



软件开发技术

第10章 面向对象设计技术

浙江大学 机械工程学院 航空制造工程研究所 王青



一、设计的目标和工作内容

二、传统方法的总体设计

三、面向对象的设计之系统设计

3.1 系统设计的目标和任务

3.2 子系统

3.3 软件架构 (Software Architecture)

3.4 框架 (Framework)

3.5 设计模式

四、面向对象设计的准则

五、对象设计



01

设计目标和工作内容

面向对象设计需要给出产品规格说明书中所规定的产品功能。更准确地说，设计工作的目标是精化分析工作的成果，直到程序员能够以某种形式实现。

在设计工作中，开发人员确定产品的内在结构，提取出方法，并分配给相应的组件（类）。特别地，必须详细制定每个方法的接口。

组件属性的提取是在分析工作中完成，但是，组件属性的格式则在设计工作中确定。

从面向对象分析到面向对象设计，是一个逐渐扩充模型的过程，尽管分析和设计的定义有明显的区别，但是在实际的软件开发过程中二者的界限是模糊的。许多分析结果可以直接映射成设计结果，而在设计过程中又往往会加深和补充对系统需求的理解，从而进一步完善分析结果。分析和设计活动是一个反复迭代的过程。

面向对象方法学在概念和表示方法上的一致性，保证了在各项开发活动之间的平滑过渡，领域专家和开发人员能够比较容易地跟踪整个系统开发过程，这是面向对象方法与传统方法比较起来所具有的一大优势。

生命周期方法学把设计进一步划分为总体设计（概要设计）和详细设计两个阶段。

相对应地，面向对象设计可再细分为系统设计和对象设计。

●系统设计

确定实现系统的策略和目标系统的高层结构

●对象设计

确定解空间的类、关联、接口形式以及实现服务的算法

02

传统方法的总体设计

传统方法的总体设计的基本目的是“总体上确定系统应该如何实现”

通过总体设计这个阶段的工作将：划分出目标系统的物理元素-程序、文件、数据库、人工过程和文档，但每个物理元素仍然处于黑盒子级，这些黑盒子内的具体内容将在以后详细设计。

总体设计的另一项重要任务是设计软件的结构，也就是确定目标系统中每个程序是由哪些模块组成的，以及这些模块的相互间的关系。

总体设计的过程

总体设计过程通常由两个阶段组成：

- 系统设计阶段：确定系统具体实现方案
- 结构设计阶段：确定软件结构

典型的总体设计过程包括如下9个步骤

(1) 设想供选择的方案

在分析结果（逻辑模型）的基础上考虑各种可能的物理实现方案，并力求选出最佳方案

具体方法经常是：从分析结果（数据流图）出发，考虑“处理”的分组方法，抛弃在技术上行不同的分组方法，余下的分组方法代表着可能的方案，并且可以启示供选择的物理系统。

(2) 选取合理的方案

从可供选择的方案中选出合理的方案

合理方案：成本/效益比合理、可以按时完成

(3) 推荐最佳方案

综合分析对比各种合理方案的利弊，推荐一个最佳方案，并为推荐的方案制定详细的实现计划

**系统设计阶段：确定
系统具体实现方案**

02 传统方法的总体设计

结构设计阶段：
确定软件结构

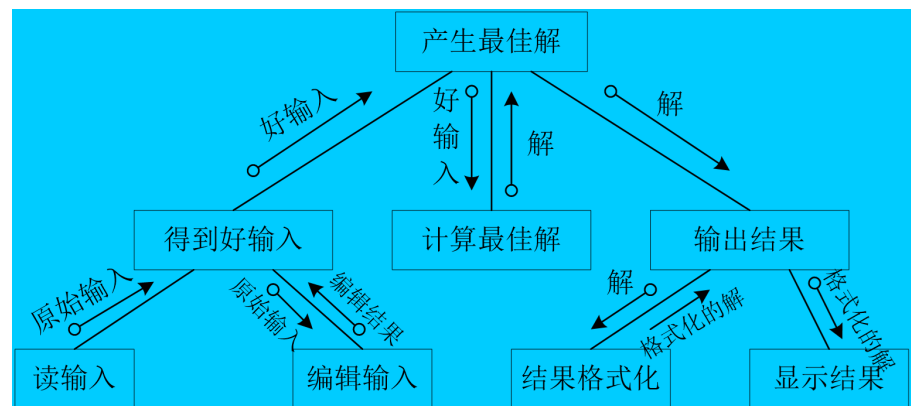
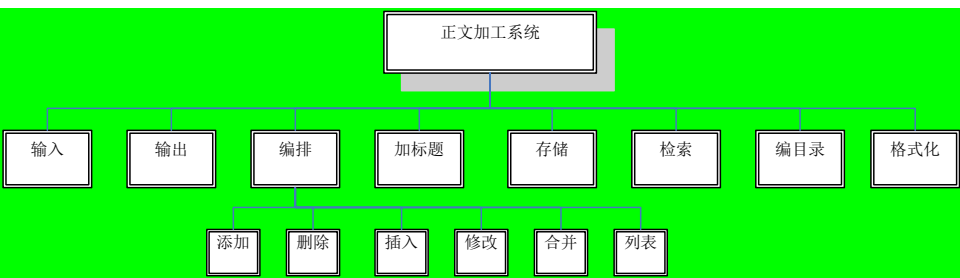
(4) 功能分解

确定系统有哪些模块组成，以及这些模块之间的关系。一般可以针对数据流图中的“处理”进行分析，对于功能复杂的“处理”进行分解，最后，使得每个“处理”对程序员来说都是“显而易见”的。将每个“处理”理解为一项功能。

(5) 设计软件结构

通过程序中的一个模块完成一个适当的子功能。应该把模块组织称良好的层次关系。顶层模块调用它的下层模块以实现程序的完整的功能，每个下层模块在调用更下层的模块从而完成程序的一个子功能，最下层的模块完成最具体的功能。

软件结构可以用层次图或结构图来描绘。



(6) 设计数据库

对于需要使用数据库的应用系统，软件工程师应该在需求分析阶段所确定的系统数据需求的基础上进一步设计数据库。

(7) 制定测试计划

在软件开发的早期阶段考虑测试问题，能促使软件设计人员在设计时注意提高软件的可测试性。

(8) 书写文档

应该用正式的文档记录总体设计的结果，在这个阶段应该完成的文档有：

- 系统说明：用系统流程图描述系统构成方案 + 组成系统的物理元素清单
- 成本/效益分析
- 对最佳方案的描述：概括、精化的数据流图、软件结构（模块层次图或结构图）、模块间的接口关系；
- 需求、功能、模块之间的交叉参照关系
- 用户手册
- 测试计划：测试策略，测试方案，预期的测试结果，测试进度计划
- 详细实现计划
- 数据库设计结果

(9) 审查和复审

03

面向对象设计之系统设计

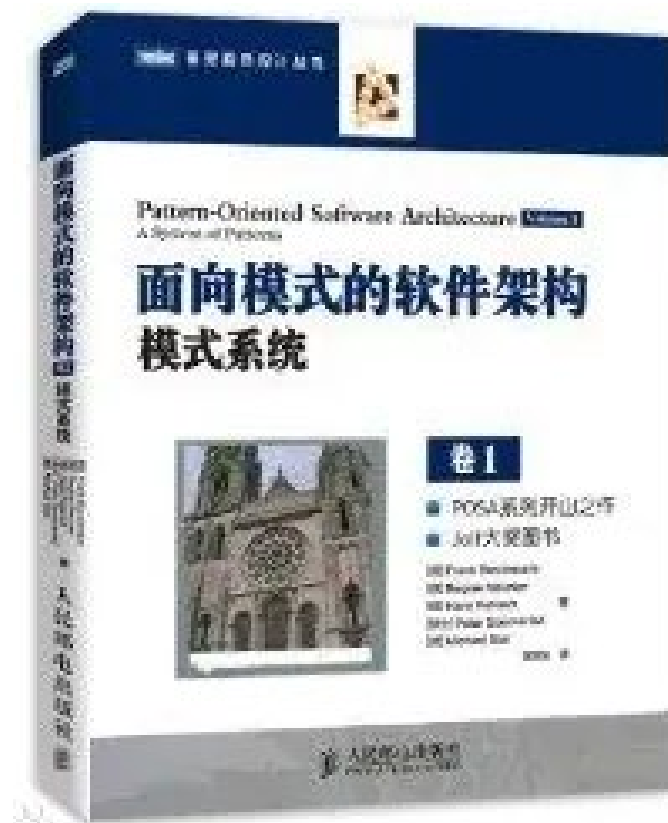
3.1 系统设计的目标和任务

确定系统的整体结构和风格，具体而言：

- 对系统进行划分，将系统划分为多个子系统。
- 各个子系统是如何组织的。

先介绍几个概念：

- 子系统
- 架构
- 框架
- 设计模式



3.2 子系统

人类解决复杂问题是普遍采用“分而治之，各个击破”的策略。同样，软件工程师在设计比较复杂的应用系统时的普遍策略也是**首先把系统分解为若干个比较小的部分，然后再分别设计每个部分**。这样做有利于降低设计的难度，有利于分工协作，也有利于维护人员对系统理解和维护。

系统的主要组成部分称为子系统。每个子系统都会以一些公共主题为基础，比如具有相似的功能、相同的物理范围或在同类硬件上的执行情况等等。

如PPT系统，可以认为它由文件、编辑、视图等等子系统构成。航天飞机的软件系统包括以下子系统：生命支持、导航、引擎控制、科学实验等。

子系统 (subsystem)不是对象、也不是函数，而是一组相关的类、关联、操作、事件和约束，并有一个与其他子系统之间的定义清晰的小型接口。

一般根据系统所提供的服务来划分子系统。所谓服务是指完成某个共同目的的一组相关功能。如处理输入/输出，绘制图形，文件管理等。

子系统汇集了一组相关的功能，这些功能完成问题的某个方面的工作。

一般每个子系统能独立设计，而不会影响其他子系统。但一个系统不能划分为太多的子系统。子系统的数目应该与系统规模基本匹配。

各个子系统之间应该具有尽可能简单、明确的接口，接口确定了交互形式和通过子系统边界的信息流，但是无需规定子系统内部的实现算法。因此可以独立地设计各个子系统。

在划分和设计子系统时，应该尽量减少子系统之间的依赖性。

大多数面向对象设计模型在逻辑上都由4大部分组成。这4大部分对应与组成目标系统的4个子系统：

- 问题域子系统
- 人机交互子系统
- 任务管理子系统
- 数据管理子系统

在软件系统中，子系统之间的交互有两种可能的方式：

➤ **客户-服务器关系 (Client-Sever; C/S)**：作为客户的子系统调用作为“服务器”的子系统，服务器完成某些服务工作后返回结果。使用这种交互方案，作为客户的子系统必须了解作为服务器的子系统的接口，然而服务器却无须了解客户的接口，因为任何交互行为都是由前者驱动的。

➤ **平等伙伴关系 (Peer to Peer)**：每个子系统都有可能调用其他子系统，因此，每个子系统都必须了解其他子系统的接口。由于各个子系统需要相互了解对方的接口，因此这种组织系统的方案比起客户-服务器方法来，子系统之间的交互更复杂，而且这种交互方式还可能存在通讯环路，从而使系统难于理解，容易发生不易觉察的设计错误。

要尽量创建客户/服务器关系的子系统交互方式，因为与点到点的双向相比，单向交互更容易构建、理解和改变。

把子系统组织成完整的系统时，有水平层次组织和垂直块组织两种方案：

◆ 层次组织

把软件系统组织成一个层次系统，每层是一个子系统。上层在下层的基础上建立，下层为实现上层功能而提供必要的服务。

每一层内所包含的对象，彼此间相互独立，而处于不同层次上的对象，彼此间往往有关联。上下层间存在客户-服务器关系。低层子系统提供服务，为服务器端，上层子系统使用下层子系统的服务为客户端。

◆ 块状组织

把软件垂直分为若干个相互独立的、弱耦合的子系统，一个子系统相当于一块，每块提供一种类型的服务。

一个完整的软件系统往往由子系统通过层次、块等各种结构组合而成。

3.3 软件架构 (Software Architecture)

软件架构与建筑类似，可以先来看一下建筑风格：

- ◆ 建筑风格等同于建筑体系结构的一种可分类的模式，通过诸如外形、技术和材料等形态上的特征加以区分。
- ◆ 之所以称为“风格”，是因为经过长时间的实践，它们已经被证明具有良好的工艺可行性、性能与实用性，并可直接用来遵循与模仿（复用）。



结构清晰、规则，遵循简单的形状和数学比例。
简单的圆柱、对称，区别于日后不规则的、复杂的多面建筑。
古典风格的圆柱、完美的几何设计和半球形的屋顶。

圣保罗大教堂



狭窄的长条形主殿被替换为宽阔的圆形；从外部看，中央剧烈的突出。较多的使用光影效果，具有强烈的对比效果，或者使用均衡的光线。较多的使用装饰品，大范围的天花板壁画，内部经常只用绘画和雕塑。



强调垂直高度；石骨架结构；

不论是墙和塔都是越往上分划越细，装饰越多，也越玲珑，且顶上都有锋利的、直刺苍穹的小尖顶；

丛生的圆柱、悬空的扶壁、有龙骨的拱顶、尖顶形状的圆弧拱门；

优美的彩色玻璃窗画；强调雕塑的细节，多采用圆雕和接近圆雕的高浮雕

软件系统同建筑一样，也具有若干特定的“风格”（software architectural style）；这些风格在实践中被多次设计、应用，已被证明具有良好的性能、可行性和广泛的应用场景，可以被重复使用；实现“软件体系结构级”的复用。

◆ 软件体系结构风格的定义：

软件体系结构是描述某一特定应用领域中系统组织方式的惯用模式，即由成功的软件系统组织结果抽象出来，并且用于不同的软件开发领域中的某一模式或者某一类模式。

架构 (Architecture)

经常也被翻译为：构架、体系结构。架构是以组件、组件之间关系、组件与环境之间的关系为内容的某一系统的基本组织结构，以及指导上述内容设计与演化的原理---IEEE610.12-1990年软件工程标准词汇。

组件：组件是广泛意义上的元素之意，并不是指和CORBA、DCOM、EJB等相关的专有的组件概念。

从组成上看：架构描述软件组成关系以及组件与组件之间的交互。

从内容看：架构包含以下重要问题的决策：

- 软件系统的组织
- 选择软件组成系统的结构元素之间的接口，以及当这些元素相互协作时所体现的行为
- 如何组织这些元素，使它们逐渐合成为更大的子系统

将目标系统划分为子系统，建立子系统的组织方式、确立子系统之间的交互方式和设计子系统之间的交互接口是软件架构的工作内容之一。（这里可以把子系统看做软件的组件）

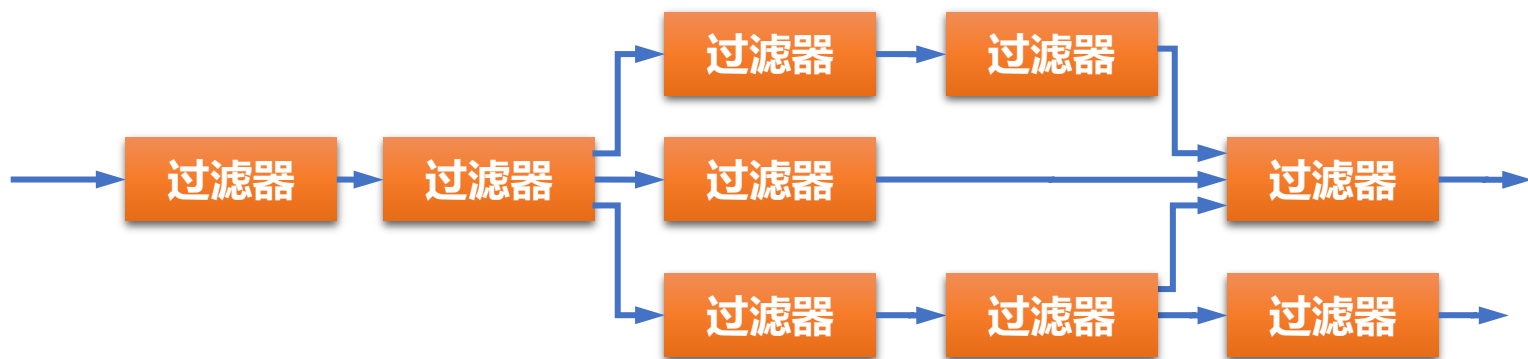
子系统也有架构，系统需要架构设计，子系统如果足够复杂，也需要架构设计，子系统作为特殊的系统（在特定的上下文中，这个系统作为更大的系统的一部分出现），它也是多个元素组成的整体，而架构规定了它是如何划分为多个部分的，以及规定了多个部分之间的交互机制和交互接口。

■ 典型体系结构风格-架构模式

- (1) 数据流风格 (Pipes and Filters模式)
- (2) 主程序/子程序体系结构 (模式)
- (3) 层次结构 (Layers模式)
- (4) 仓库风格 (Blackboard模式)
- (5) 客户/服务器结构 (Client-Server模式)
- (6) 浏览器/服务器结构 (模式)
- (7) 模型-视图-控制器 (Model-View-Controller模式)
- (8) 微内核 (Microkernel) 架构

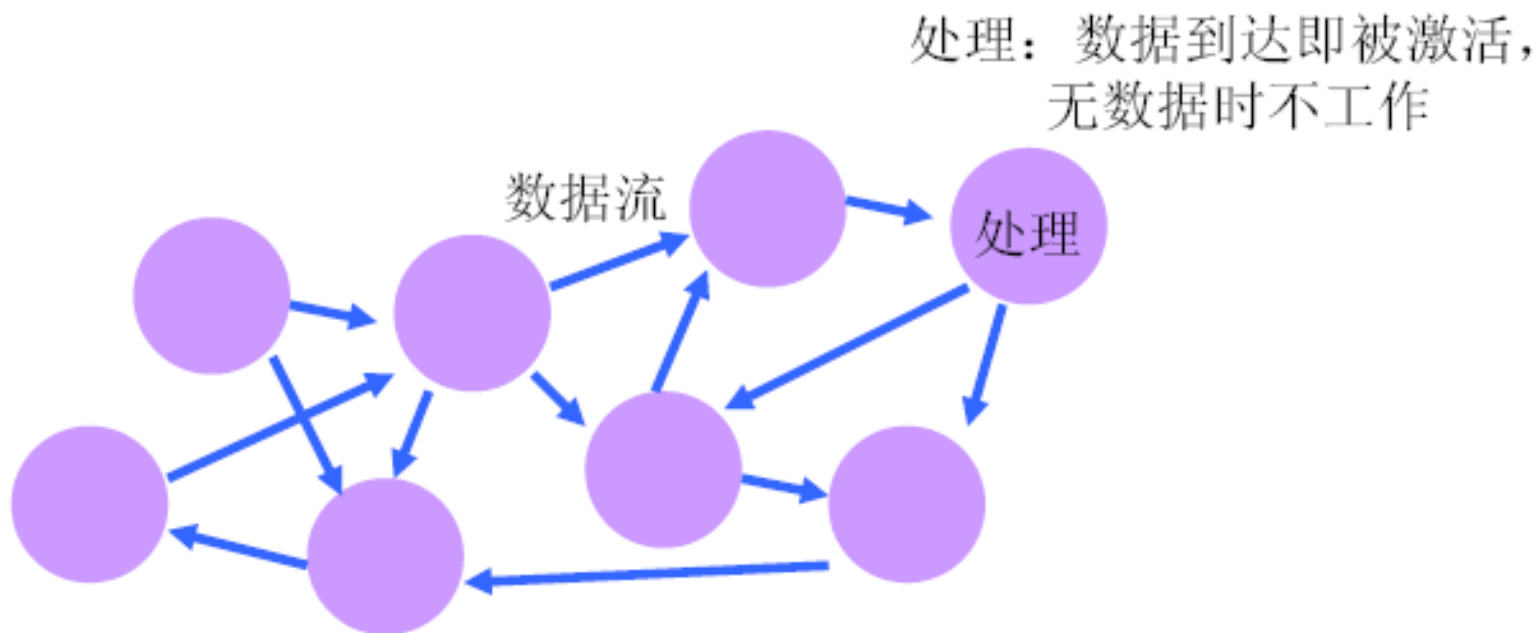
(1) 数据流风格==管道和过滤器结构 (Pipes and Filters模式)

当输入数据经过一系列的计算和操作构件的变换形成输出数据时，可以应用这种体系结构。拥有一组称为过滤器（filter）的构件，这些构件通过管道（pipe）连接，管道将数据从一个构件传送到下一个构件。每个过滤器独立于其上游和下游的构件而工作。过滤器的功能就是针对某种输入产生某种输出。它没有必要了解其相邻的过滤器的工作。



当数据流退化为单线的变换，则称为批处理序列。

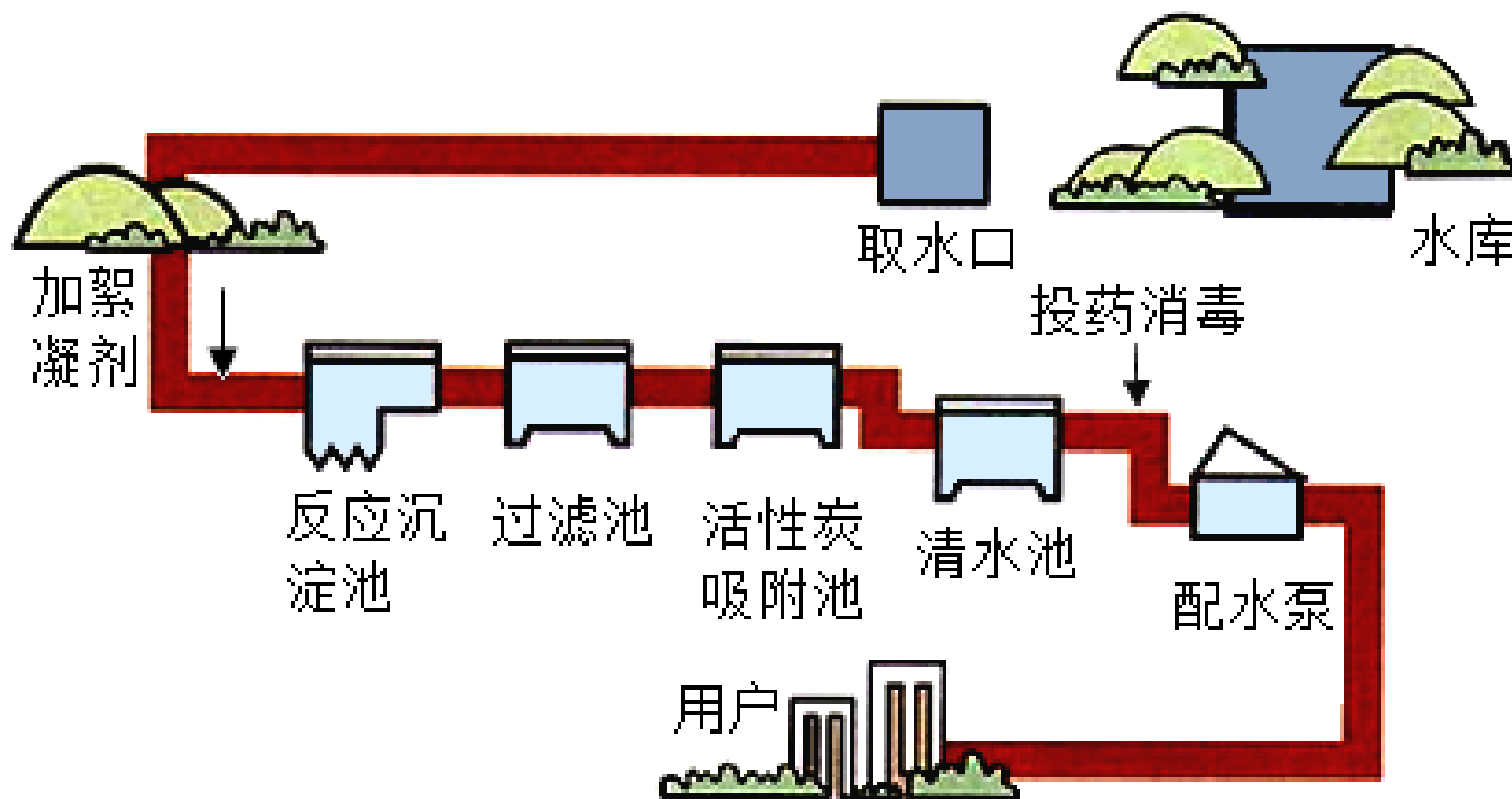
数据流风格的直观理解：



Three Examples of Data Flow

- ❑ Pipe-and-Filter (管道/过滤器)
- ❑ Batch Sequential (批处理序列)
- ❑ Process Control (过程控制)

自来水生成的示意图：



管道-过滤器风格的例子 - 编译器

输入：源程序

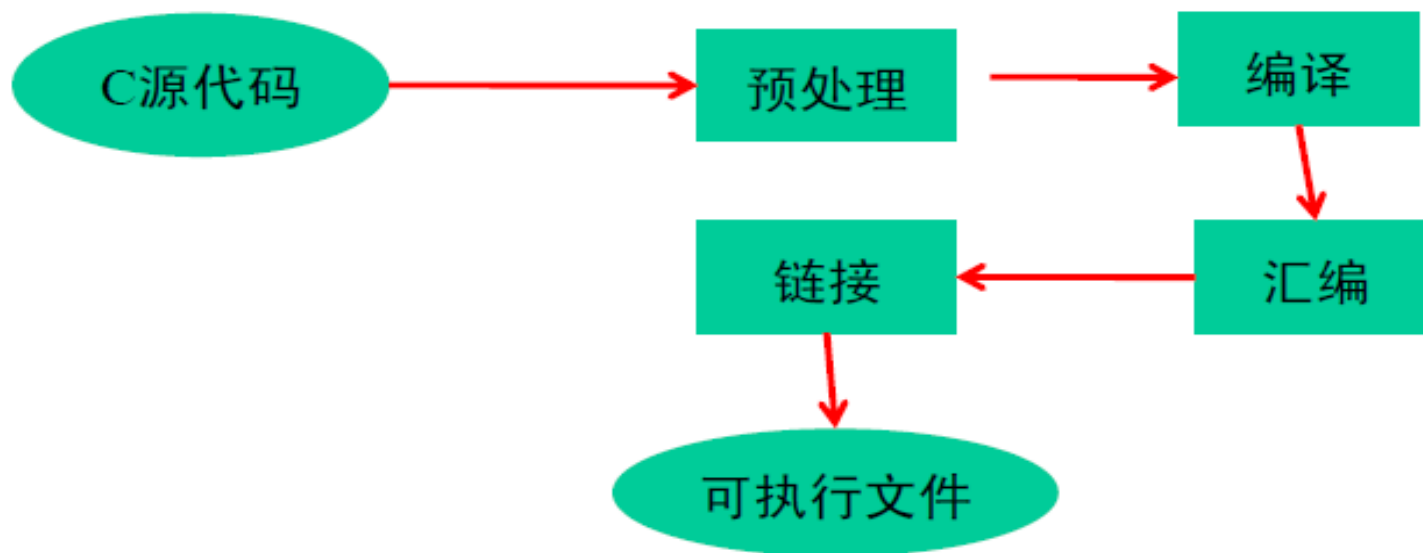
预处理：负责宏展开和去掉注释等工作。

编译：进行词法分析、语法分析、语义分析、代码优化和代码产生。

汇编：负责把汇编代码转换成机器指令，生成目标文件。

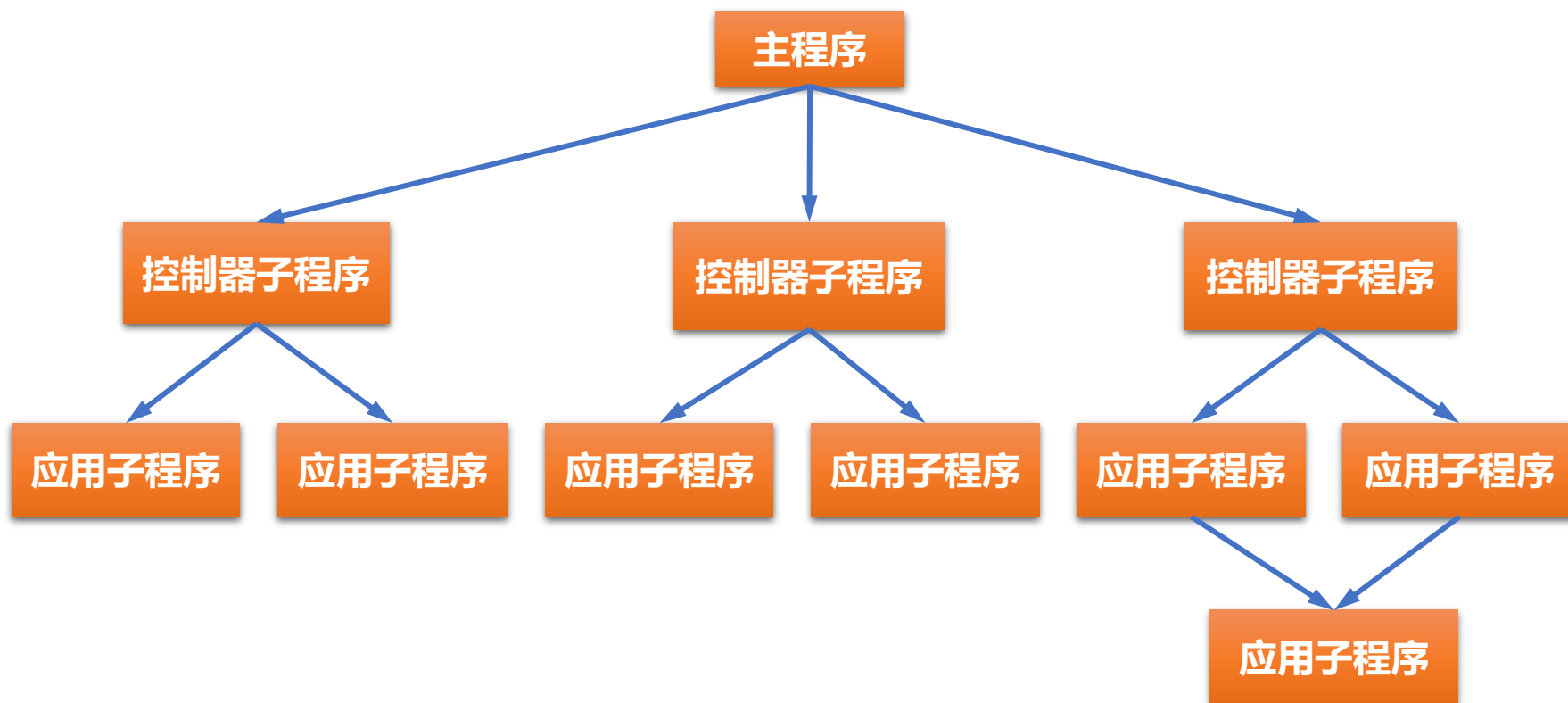
链接：负责把多个目标文件、静态库和共享库链接成可执行文件/共享库。

输出：可执行文件/共享库。

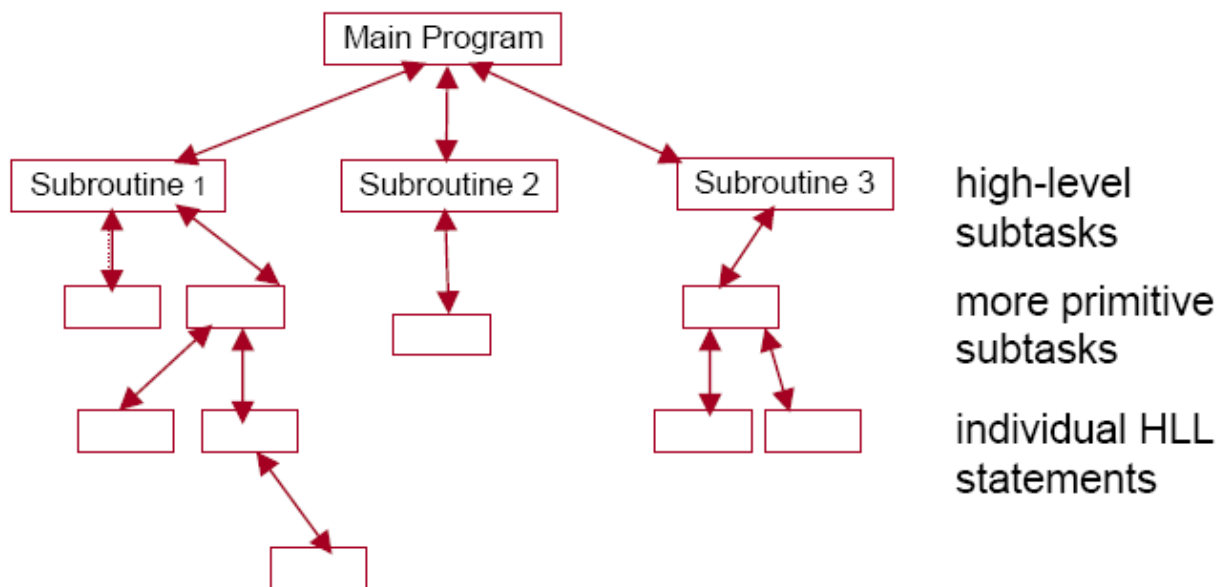


(2) 主程序/子程序体系结构

传统的程序结构图。

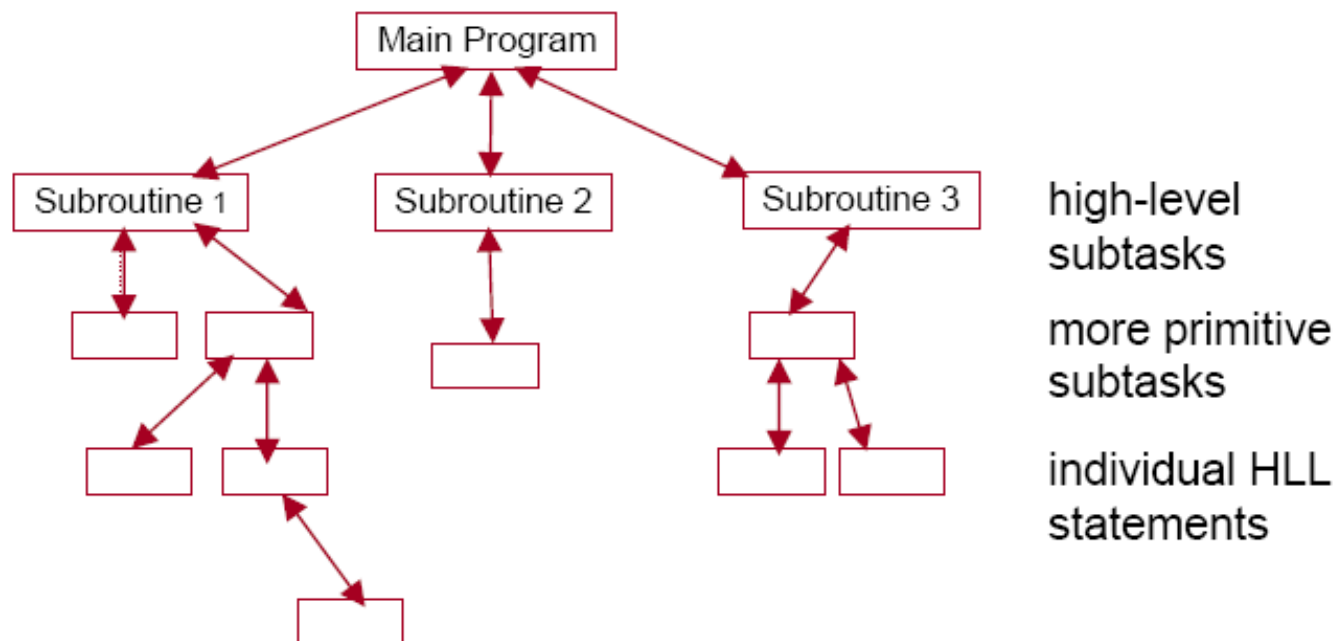


结构化程序：自上而下的功能设计 (Top-down functional design)



结构化程序：逐层分解 (Hierarchical decomposition)

- 从功能的观点设计系统
- 结构化设计与逐步细化
- 逐步分解
- 基于“定义—使用”关系
- 用过程调用作为交互机制
- 主程序的正确性依赖于它所调用的子程序的正确性



主程序-子程序风格的基本构成

- 构件：主程序、子程序
- 连接器：调用-返回机制
- 拓扑结构：层次化结构
- 本质：将大系统分解为若干模块(模块化)，主程序调用这些模块实现完整的系统功能。

主程序-子程序风格的优点与缺点：

◆ 优点：

已被证明是成功的设计方法，可以被用于较大程序

◆ 缺点：

- 程序超过10万行，表现不好

- (程序太大，开发太慢，测试越来越困难)

关键问题：如何对系统进行分解

主程序-子程序风格的基本构件是一个个大小不一的子程序；那么，就存在如下问题：

- 如何将一个大系统分解为若干个子程序？

- 分解的标准是什么？

- 子程序如何聚合为模块？

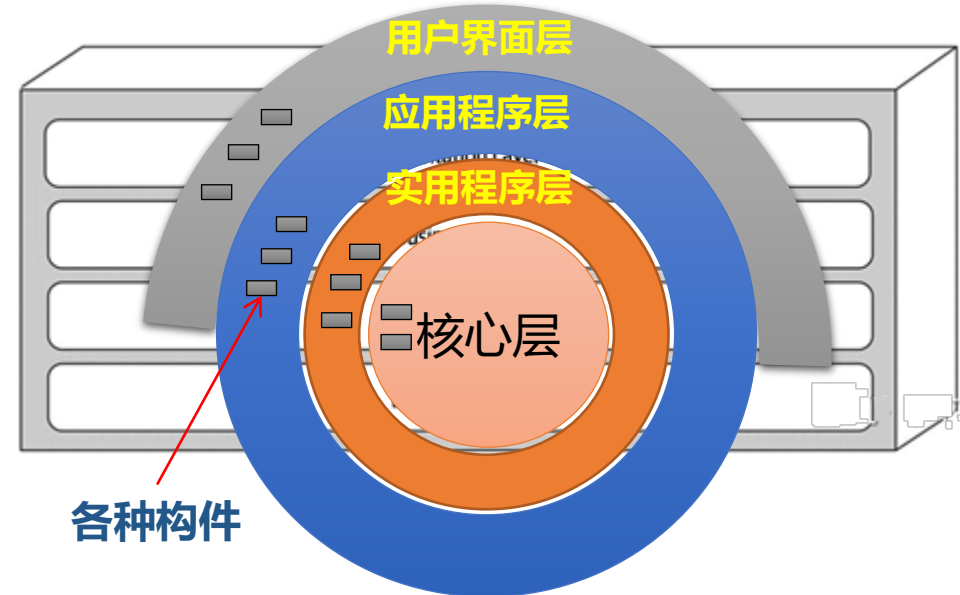
(3) 层次结构

整个系统被组织为一个层次结构，每一层为上层提供服务，并作为下一层的客户。在一些层次系统中，除精心挑选的输出函数外，其他只对相邻的层可见。从外层到内层，每层的操作逐渐接近机器的指令集。在最外层，构件完成界面层的操作；在最内层，构件完成与操作系统的连接，中间层提供各种实用程序和应用软件的功能。

一个分层系统采用多个层次组织。在这种类型的系统下，每一层必须起两个作用。

- 首先，它必须为结构中的上一层提供的服务。
- 其次，它必须以用户的身份调用下层的功能。

此外，整个系统中所有的层必须同时满足这两个要求。其中，最高一层已经没有任何更高的层，它不需要提供任何服务；最低一层没有下层，也就不会使用其他服务。

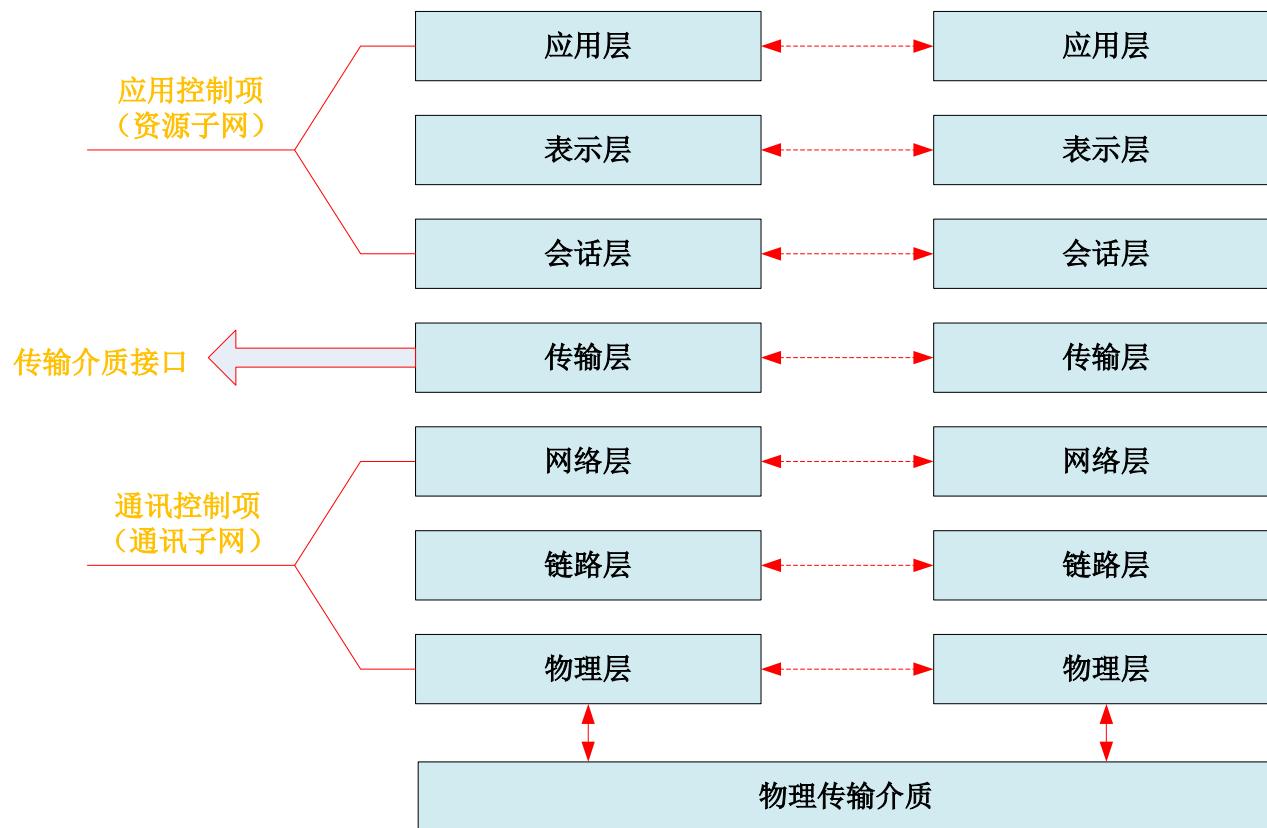




分层风格实例：计算机网络的设计

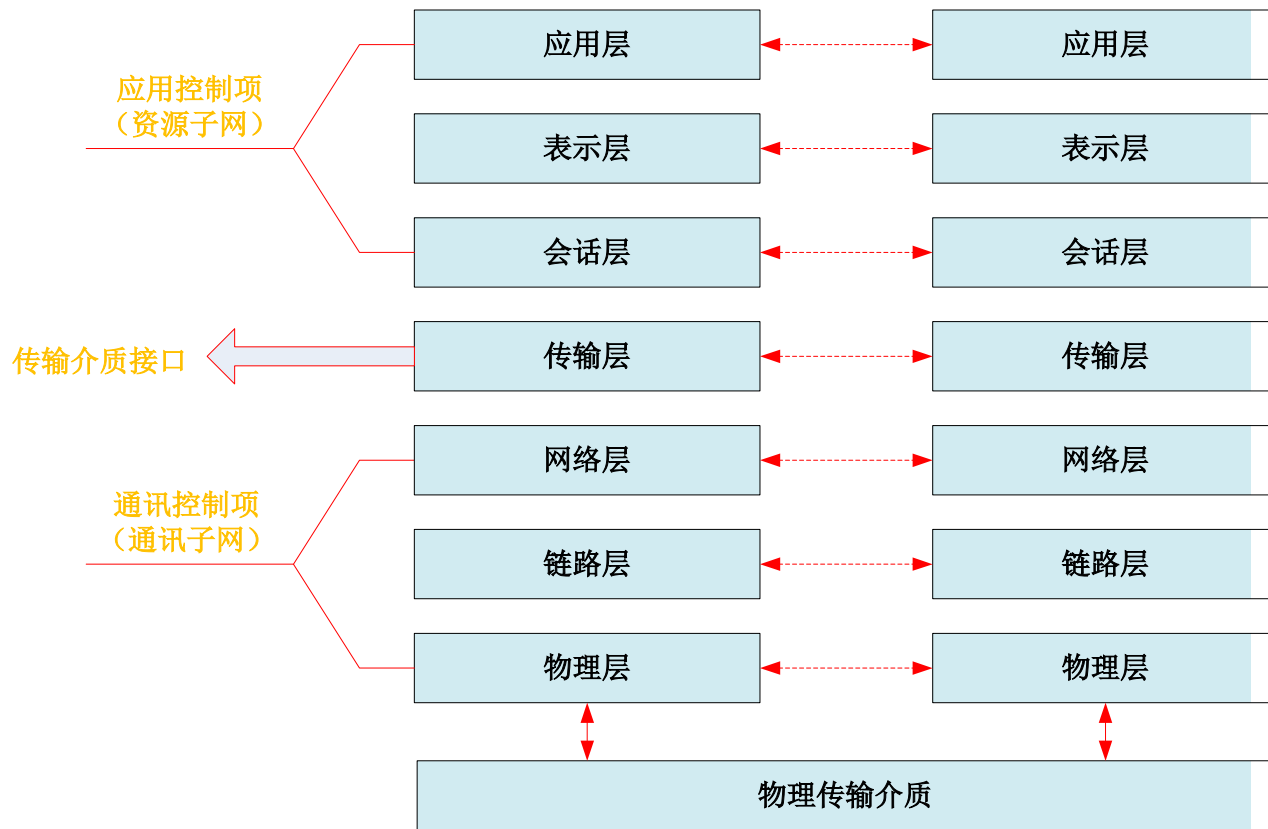
概述

网络协议设计者将计算机网络中的各个部分按其功能划分为若干个层次 (Layer)，其中的每一个层次都可以看成是一个相对独立的黑箱、一个封闭的系统。用户只关心每一层的外部特性，只需要定义每一层的输入、数据处理和输出等外部特性。



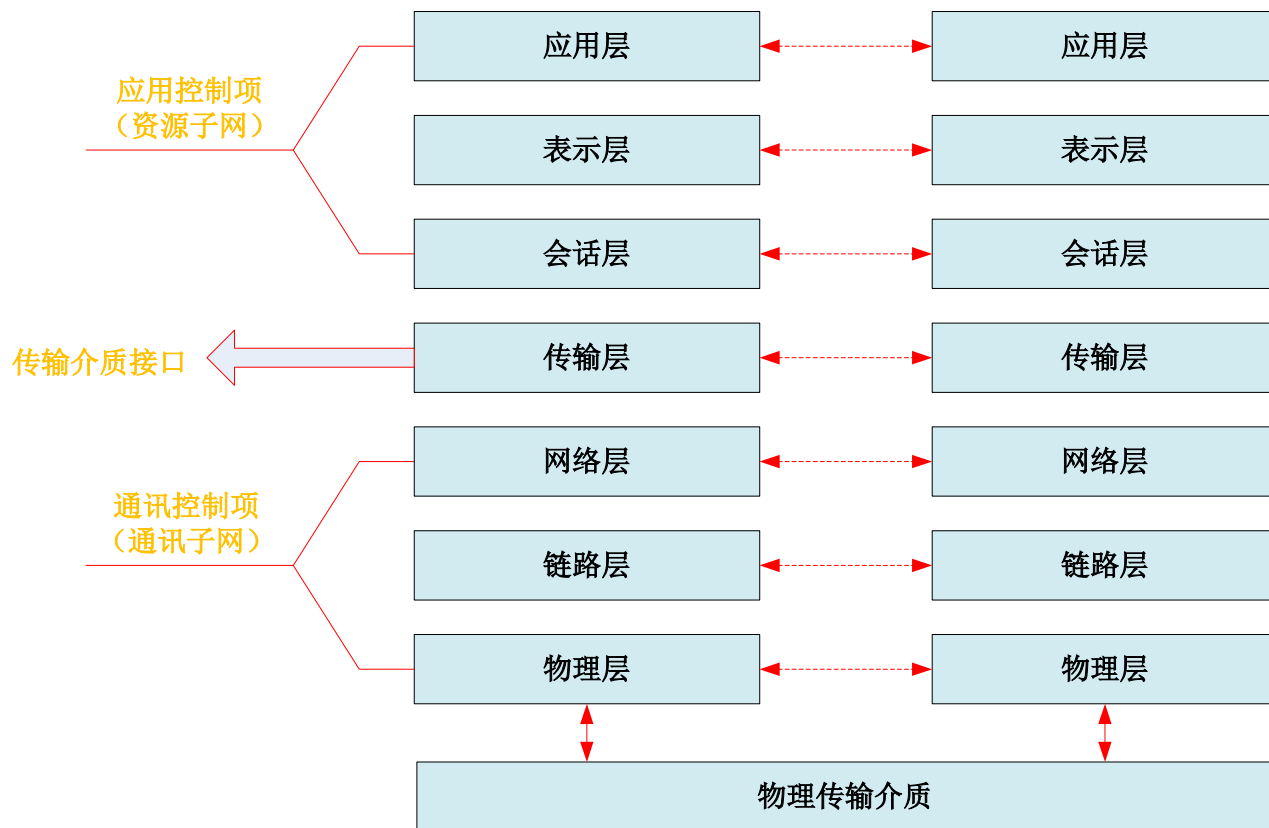
ISO/OSI采用了7层体系结构，从高到低分别是：应用层、表示层、会话层、传输层、网络层、数据链路层和物理层，如图所示。

其最高层为第7层应用层，用于同应用服务之间交换数据；最低层为第1层物理层，用于连接物理传输介质实现真正的数据通信。



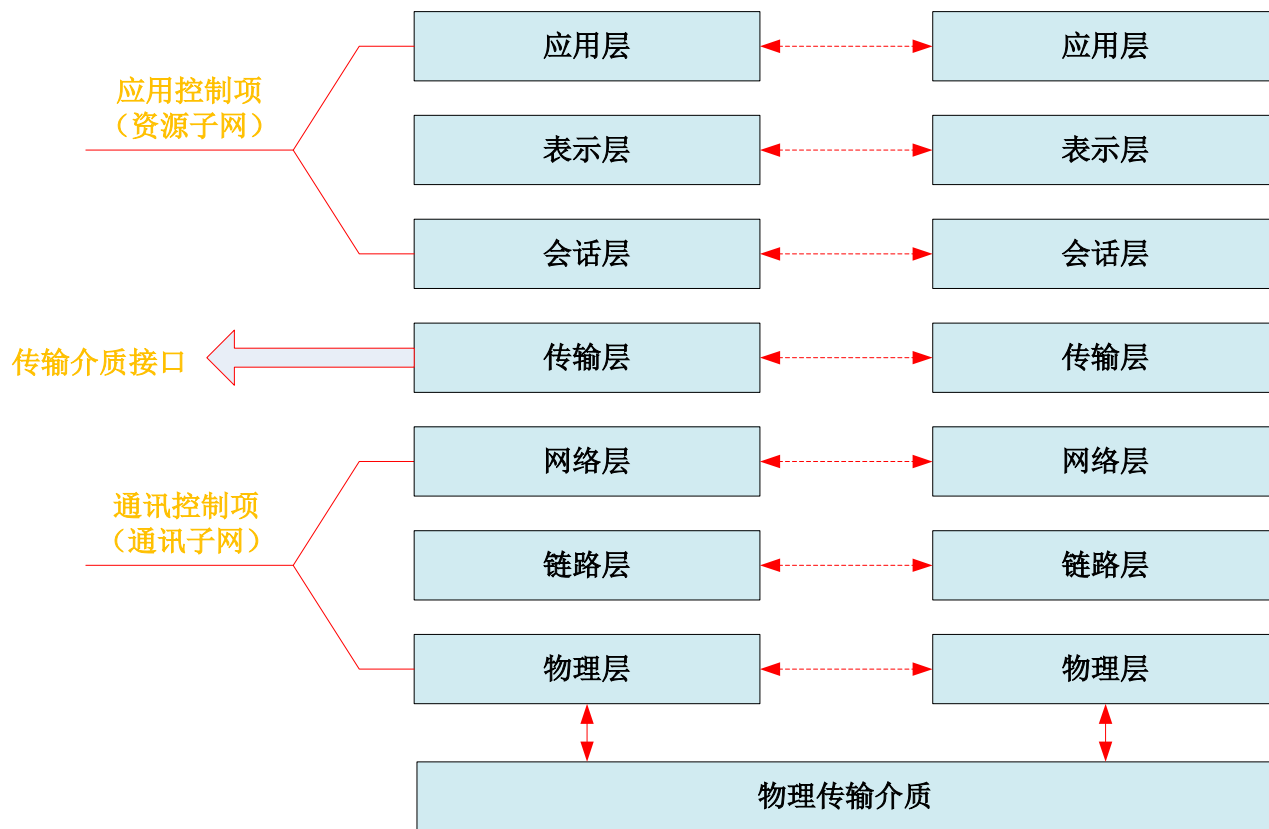
□ 层与层之间的联系是通过各层之间的接口来实现的，上层通过接口向下层提出服务请求，而下层通过接口向上层提供服务。

□ 两台计算机通过网络进行通信时，只有两物理层之间能够通过媒体进行真正的数据通信，其余各对等层之间均不存在直接的通信关系，各对等层之间只能通过各对等层的协议来进行虚拟通信。



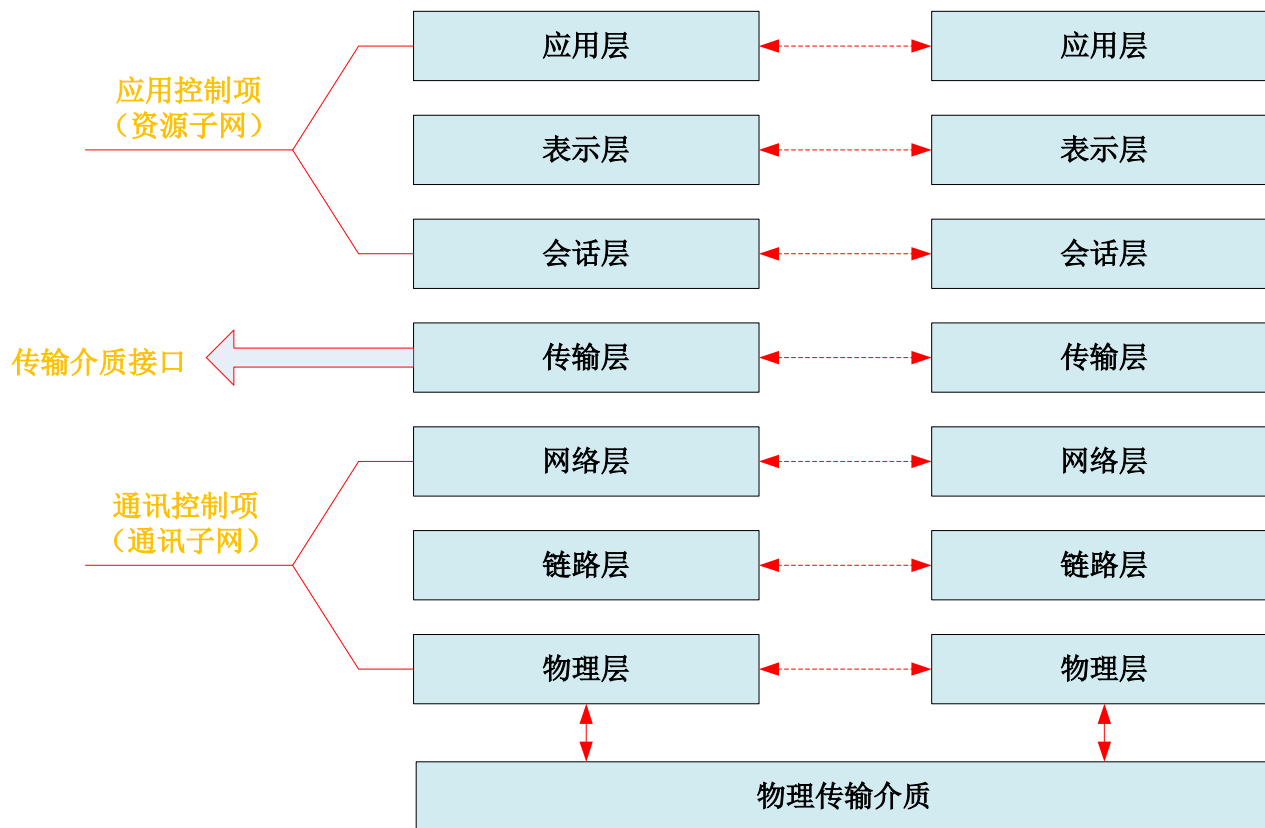
以运输公司进行货物运输为例来进行说明。

- 第1~3层3个层次相当于由运输公司负责的货物运输过程中的具体细节、具体操作方式；
- 第4层相当于运输公司与用户之间的接口；
- 第5~7层3个层次相当于由用户负责的将货物交给运输公司所需要做的准备工作。



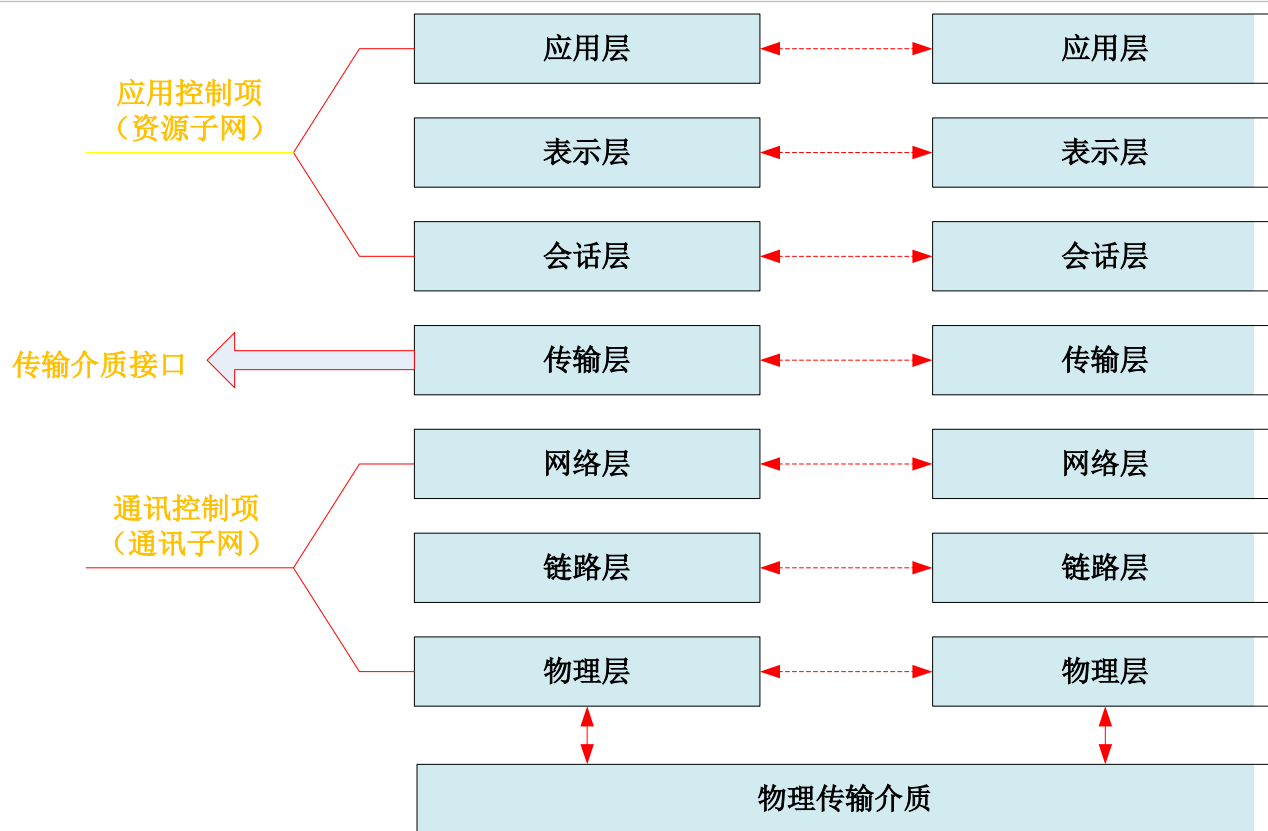
第1层是物理层 (Physical Layer)，它负责在物理信道上传输原始的数据bit流。它应该提供为建立、维护和拆除物理链路连接所需的机械的、电气的、功能和规程的特性。

这类似于运输车辆只需要负责将装在车辆内的货物（类似于bits）运送到某地就行了。



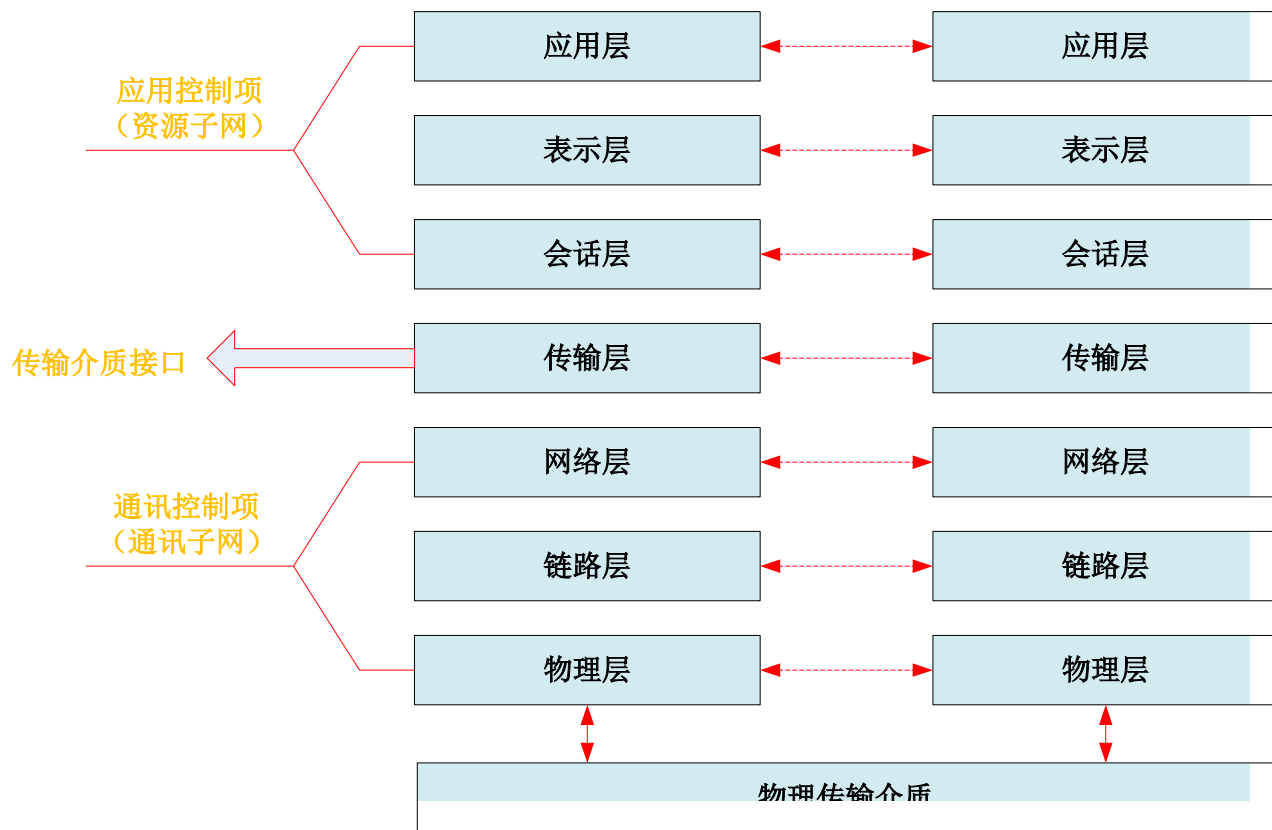
第2层是数据链路层 (Data Link Layer)，它的主要功能是纠错和流量控制，负责在可能出现差错的物理线路中实现无差错的数据传送。它应该在物理层的基础上，建立相邻结点之间的数据链路，通过差错控制提供数据帧 (Frame) 的无差错传输，并进行数据流量控制。

这类似于运输公司的运输管理和质量监督部门，需要负责在可能出现问题的运输线路之中保质保量地完成运输任务。



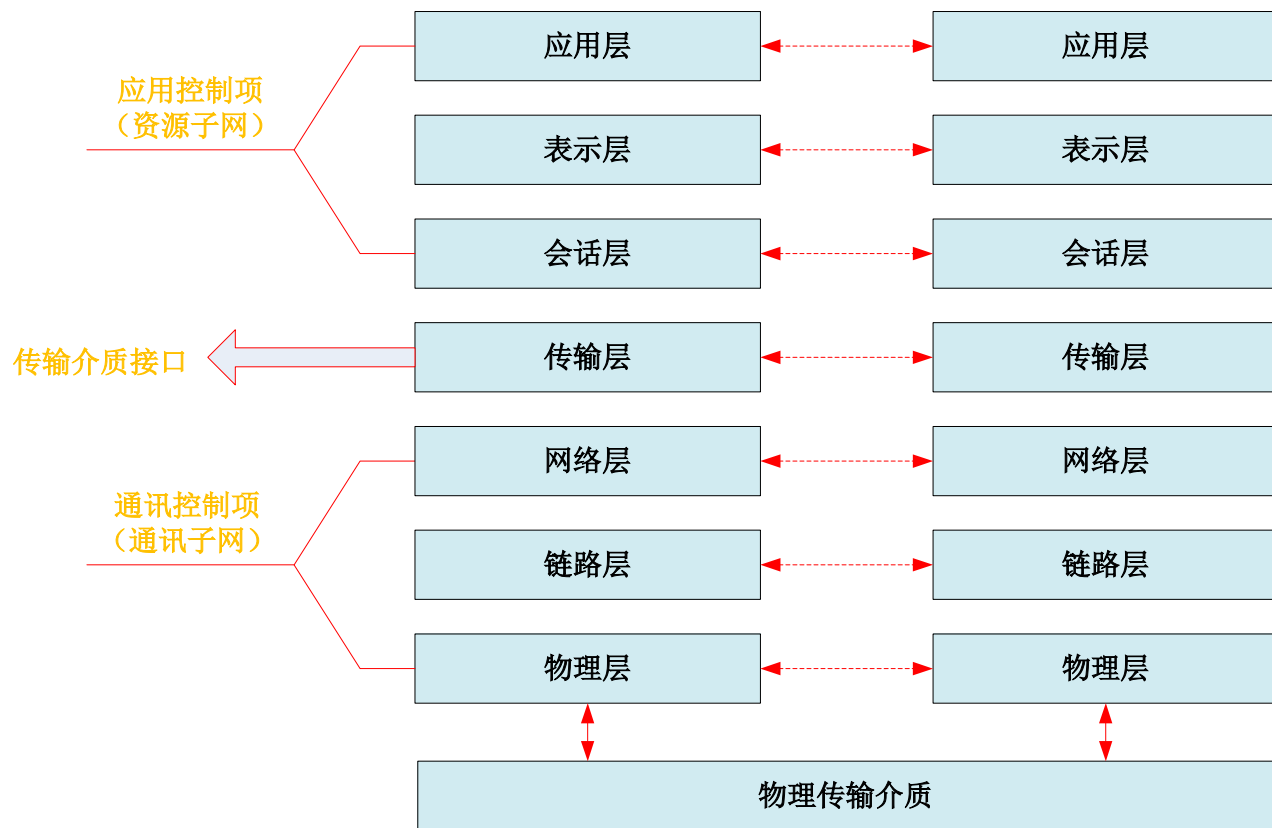
第3层是网络层 (Network Layer)，它的主要功能是路由控制（找路）、拥塞控制和数据打包。它应该为其上一层传输层的数据传输提供建立、维护和终止网络连接的手段，把上层传来的数据分割成一个一个的数据包（Packet，也叫报文分组）在结点之间进行交换传送，并且负责路由控制和拥塞控制。

这类似于运输公司需要将用户发送的货物进行分割打包，并在现有的交通网络之中负责找出一条从源地址到目的地址的线路（即找路），在找路时需要考虑到能否到达、拥塞状况、安全可靠、甚至交通费用等诸多方面的因素。



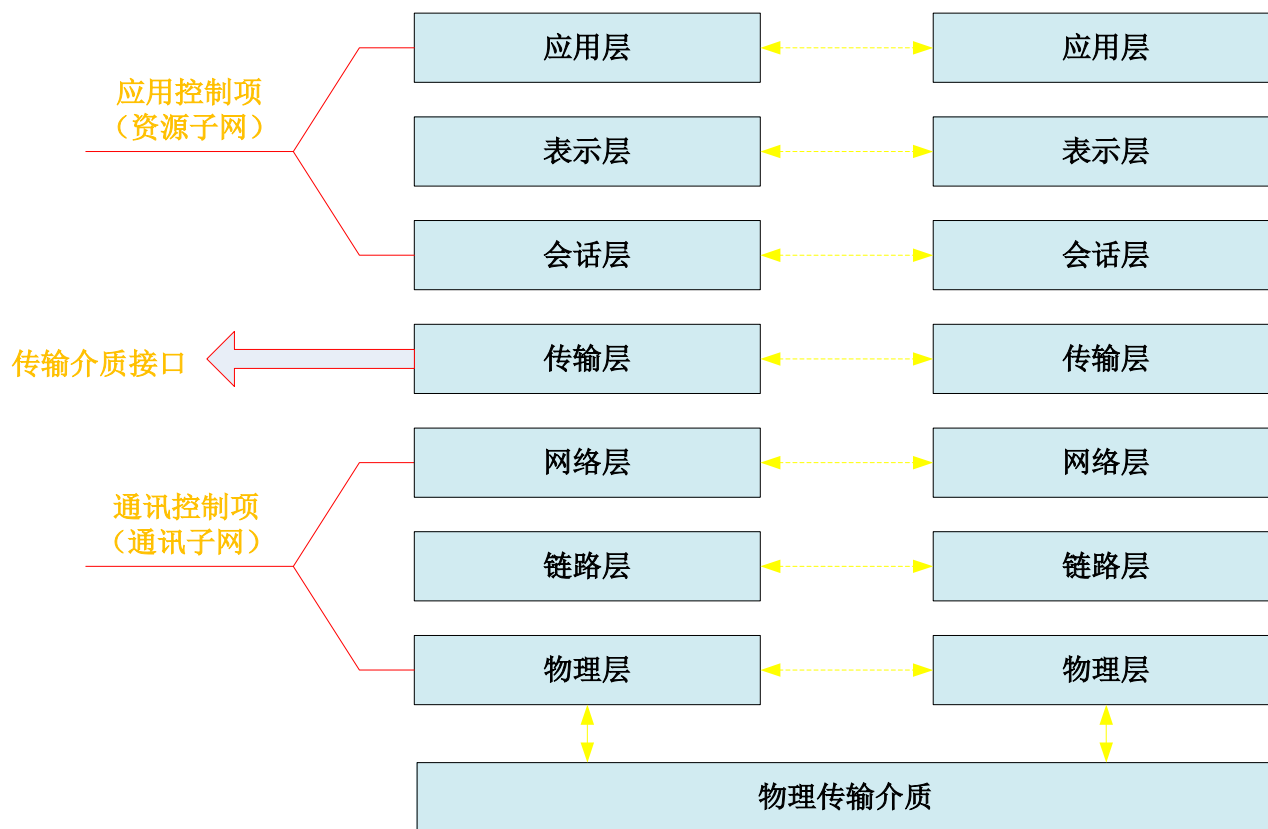
第4层是传输层 (Transport Layer)，它的主要功能是在上层和下层之间起到一种接口的功能。它应该为上层提供端到端（最终用户到最终用户）的、透明的、可靠的数据传输服务。所谓透明的传输是指在通信过程中上层可以将下面各层看作是一个封闭的黑箱系统，传输层对上层屏蔽了传输系统的具体细节。

这类似于运输公司在各个地方设置的业务接洽处，它负责在用户和公司之间建立起一个货物交接的桥梁，使得用户不用去管运输公司将以什么样的方式将货物运送到目的地，也就是说业务接洽处对用户屏蔽了货物运输中的具体细节。



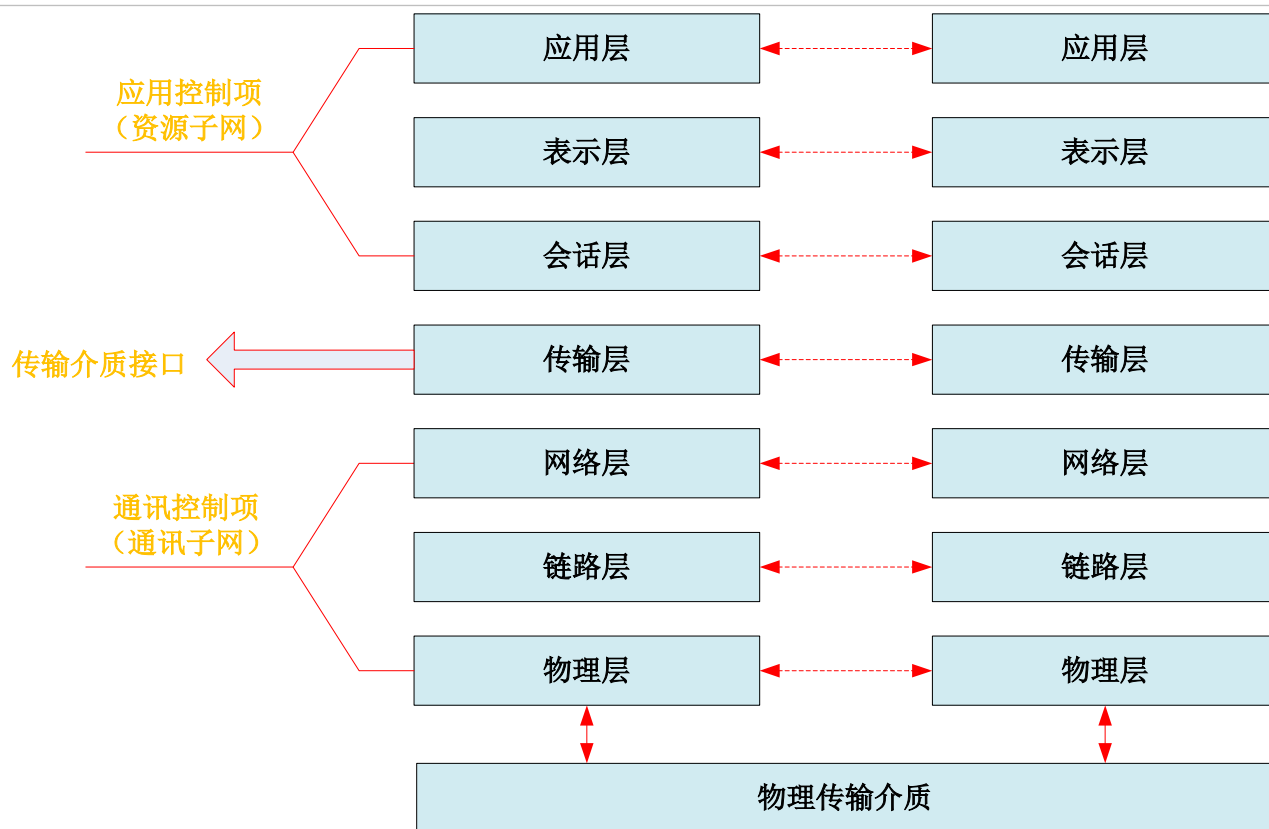
第5层是会话层 (Session Layer)，它的主要功能是负责收发数据的交接工作、并组织和管理数据。它应该为表示层提供建立、维护和结束会话连接的功能，并提供会话管理服务。

这类似于用户公司的货物收发室，它负责与运输公司打交道，完成用户公司货物的收发的交接工作、并组织管理公司内部要收发的货物。



第6层是表示层 (Presentation Layer)，它的主要功能是为数据提供收发、存放的具体格式和规范。它应该为应用层提供信息表示方式的服务，如数据格式的变换、文本压缩、加密技术等。

这类似于用户公司的货物收发员，它负责与用户公司内部要收发货物的部门或个人打交道，在收集要发送的货物时告诉用户应该怎样填写发货资料，在向用户发放货物时告诉用户应该完清哪些具体手续，等等。

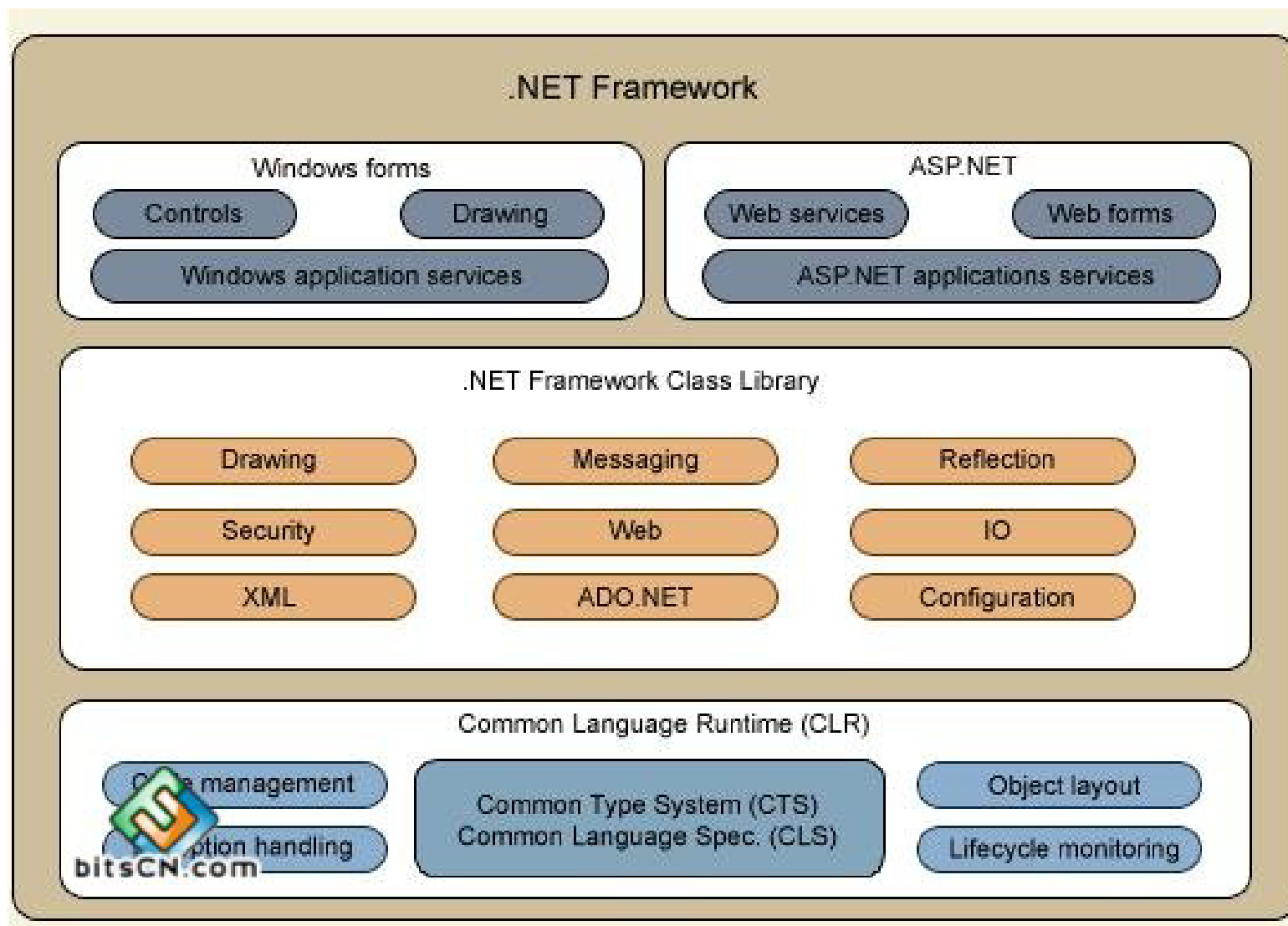


第7层是应用层 (Application Layer)，它的主要功能是为数据提供各种可行的收发方式。它应该为网络用户或应用程序提供各种应用服务，如文件传输、电子邮件 (E-mail)、分布式数据库、网络管理等。

这类似于用户公司内部的部门或个人在收发货物时，都必须遵循用户公司内部的相关规定，只能使用用户公司所允许的方式来收发货物。从另一方面来说，用户公司也要为公司内部的部门或个人收发货物提供各种可行的收发方式，让用户公司内部的部门或个人知道他们能够使用哪些方式来收发货物。



.Net 框架分层模图



(4) 仓库风格

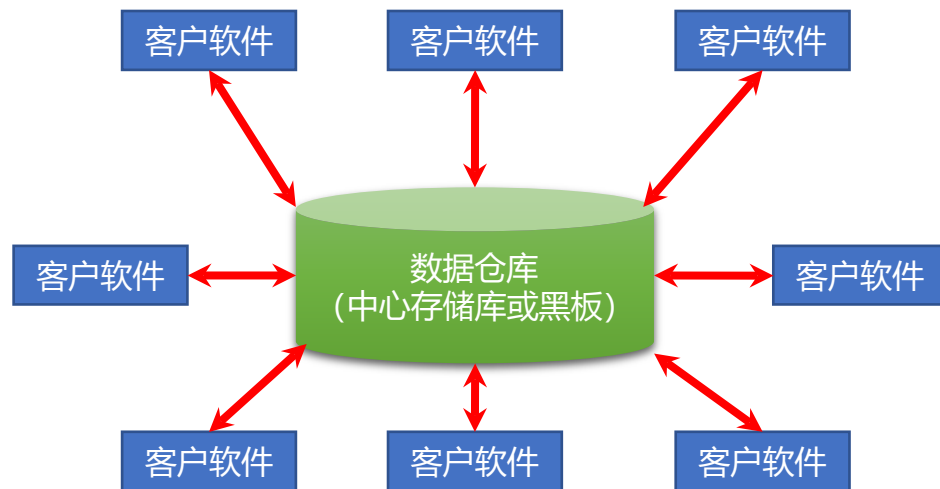
在这种风格中，数据仓库（如文本、数据库）位于体系结构的中心，其他构件会经常访问数据仓库，并对仓库中的数据进行增加、删除、修改操作。

数据库系统、超文本系统和黑板系统都属于仓库风格。

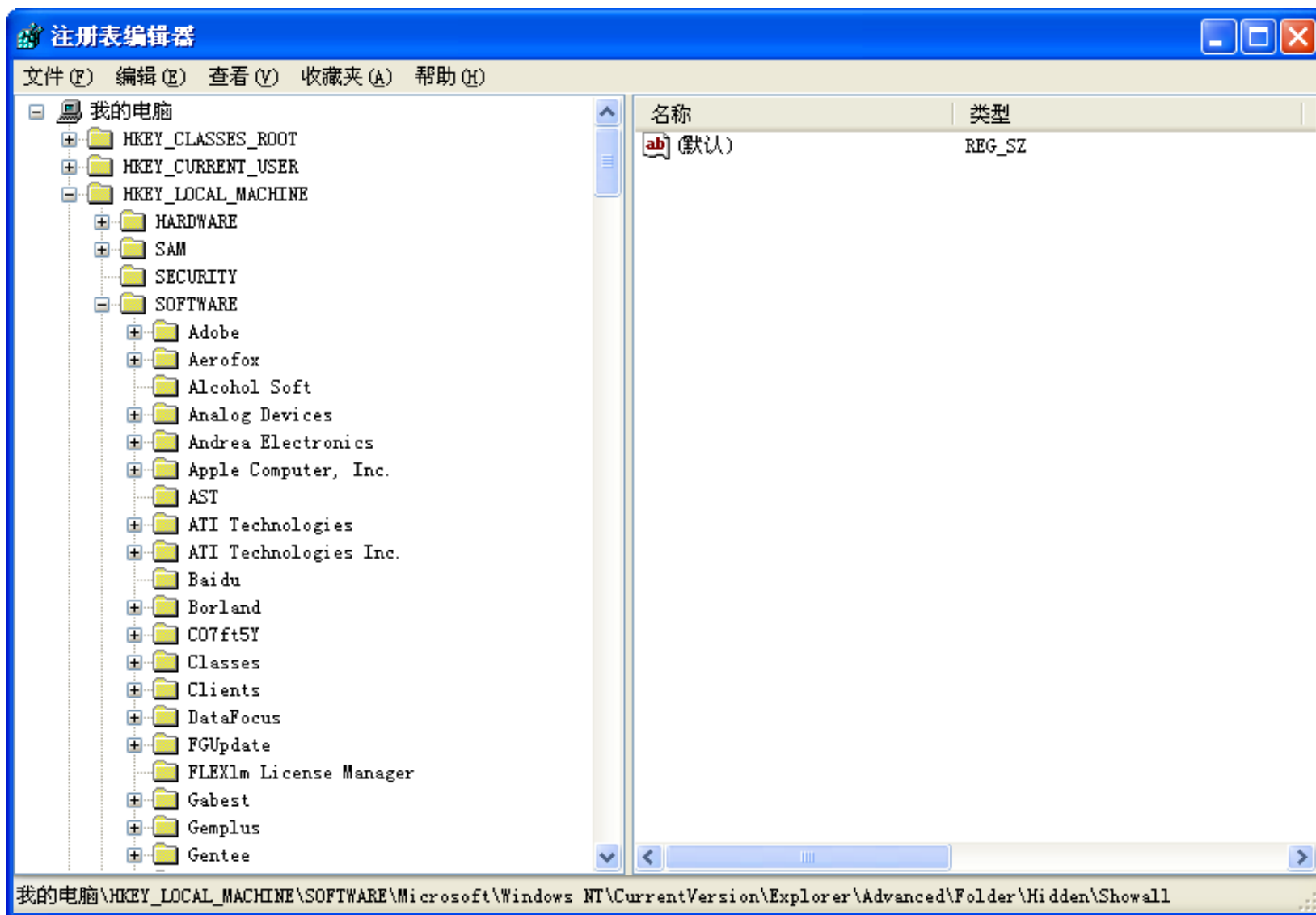
图示为一个典型的仓库风格的体系结构。

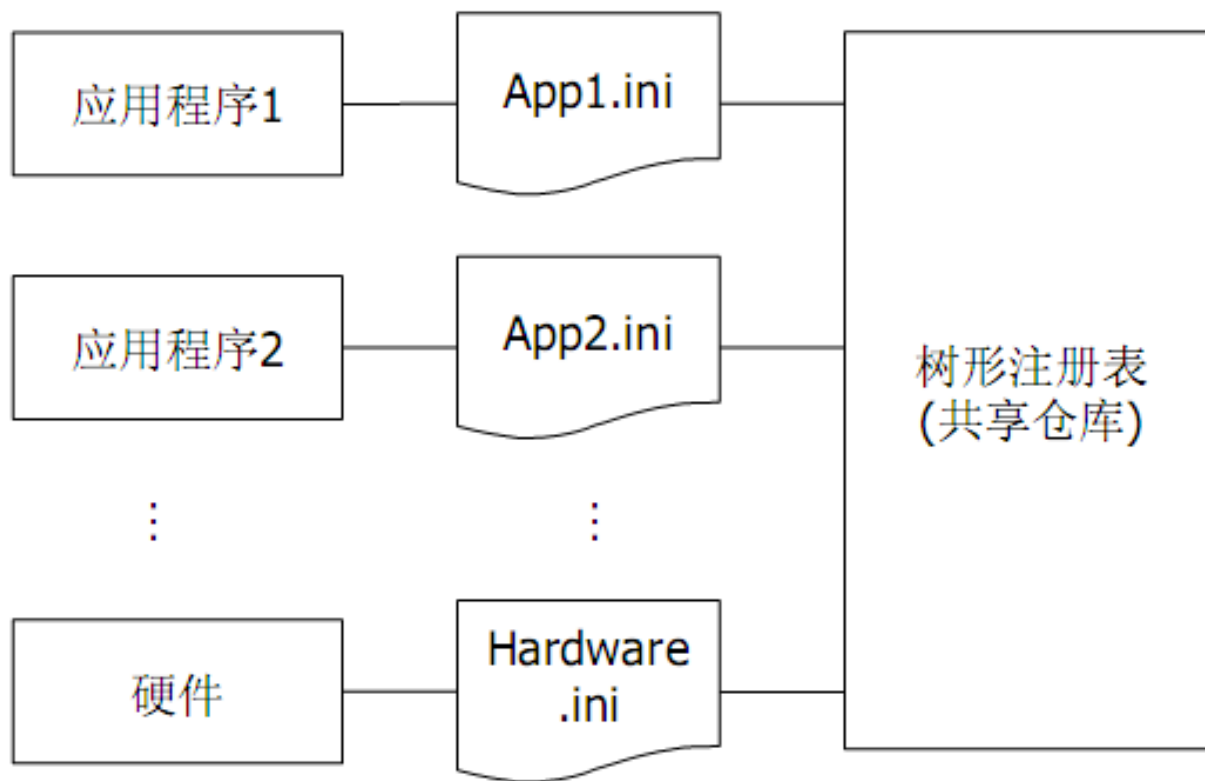
其中客户软件访问中心仓库，在某些情况下，仓库是被动的，也就是说，客户软件独立于数据的任何变化或其他客户软件的动作而访问数据==相当于数据库系统。

当将中心存储库变成黑板，它负责协调信息在客户间的传递，当用户感兴趣的数据发生变化时，它将通知客户软件===黑板系统=》主要用于信息处理领域（专家系统）。



例1：注册表(Windows Registry)



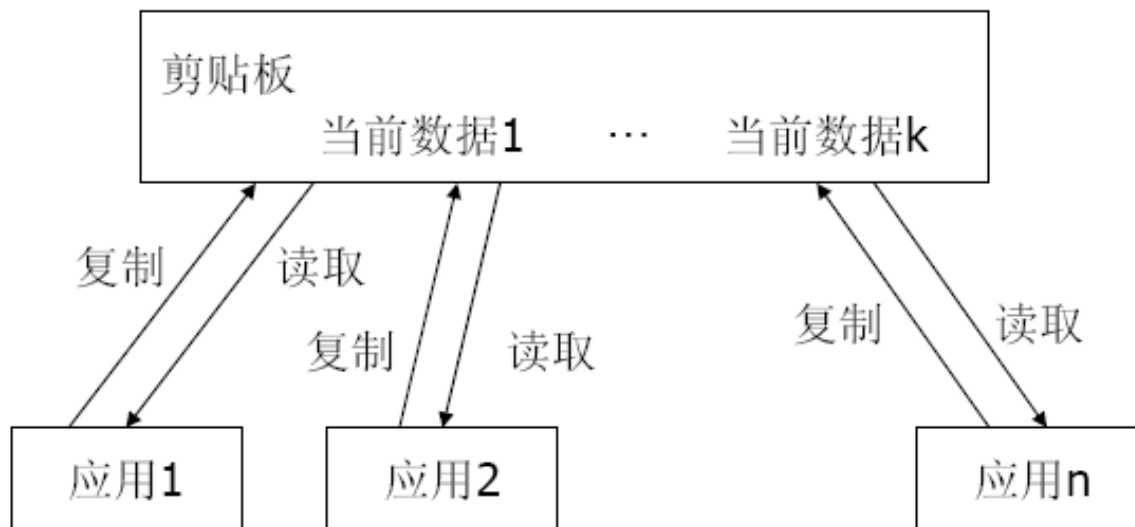


最初，硬件/软件系统的配置信息均被各自保存在一个配置文件中(.ini)；
这些文件散落在系统的各个角落，很难对其进行维护；
为此，引入注册表的思想，将所有.ini文件集中起来，形成共享仓库，为系统运行起到了集中的资源配置管理和控制调度的作用。

注册表中存在着系统的所有硬件和软件配置信息，如启动信息、用户、BIOS、各类硬件、网络、INI文件、驱动程序、应用程序等；

注册表信息影响或控制系统/应用软件的行为，应用软件安装/运行/卸载时对其进行添加/修改/删除信息，以达到改变系统功能和控制软件运行的目的。

例2：剪贴板(Clipboard)



**剪贴板是一个用来进行短时间的数据存储并在文档/应用之间进行数据传递和交换的软件程序；
用来存储带传递和交换信息的公共区域(形成共享仓库)；
不同的应用程序通过该区域交换格式化的信息；
访问剪贴板的方式：copy & paste.**

(5) 客户/服务器结构

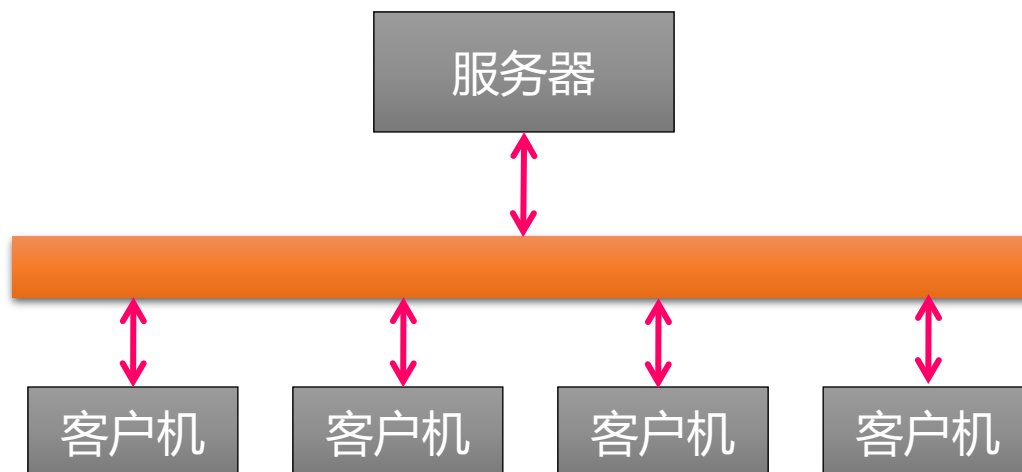
包括三个主要组成部分：

服务器：负责给其它子系统提供服务

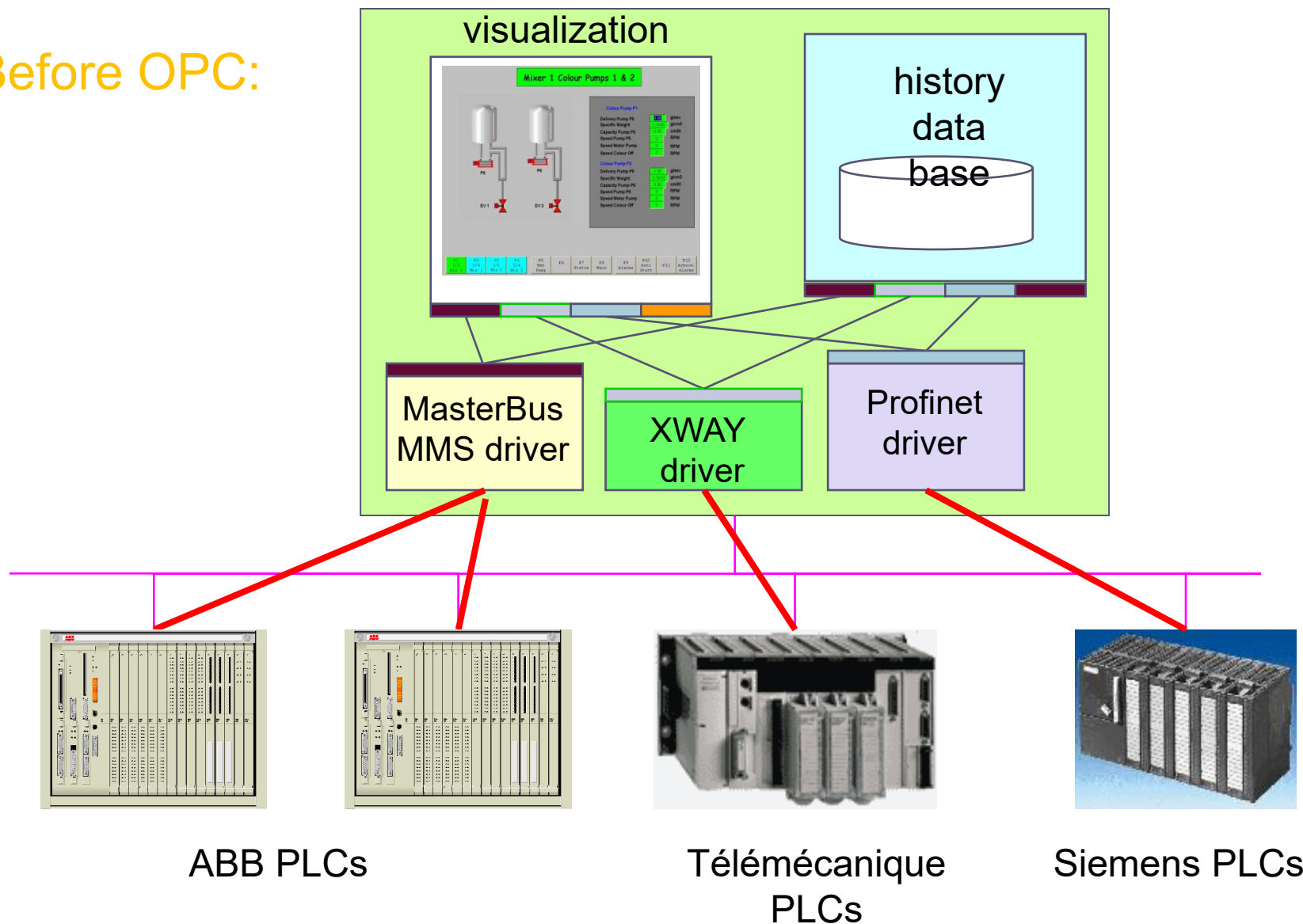
客户机：向服务器请求服务

网络：连接客户机与网络

C/S体系结构中，客户机可以通过远程调用来获取服务器提供的服务，因此客户机必须知道可用的服务器名称及它们所提供的服务，而服务器不需要知道客户机的身份。



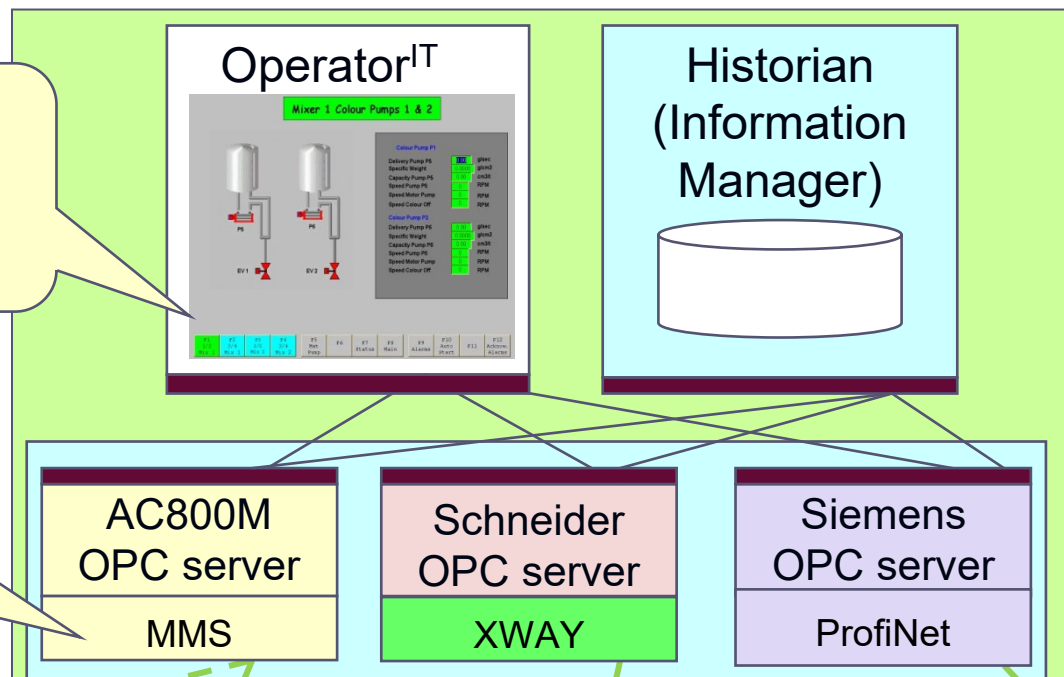
Before OPC:



客户/服务器结构实例---OPC接口技术

application software is written independently from the type of controller

the drivers still exist, but the clients do not see them anymore

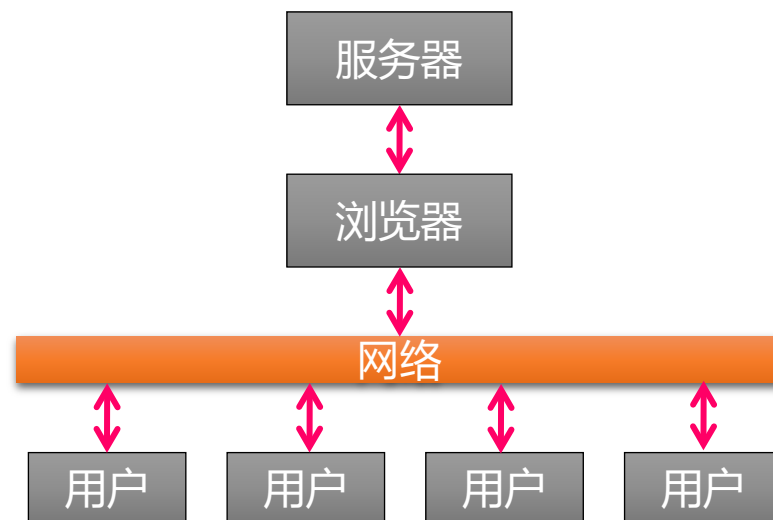


(6) 浏览器/服务器结构 (B/S结构)

是一种三层的C/S结构，增加了一个应用服务层（浏览器）。

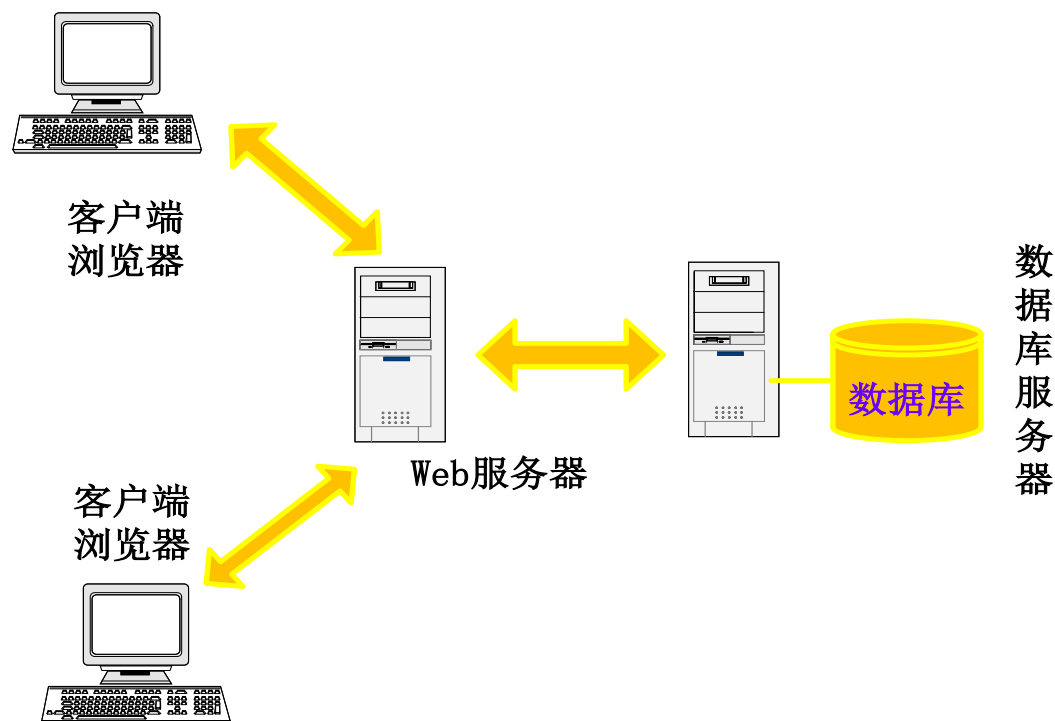


三层C/S结构



B/S结构

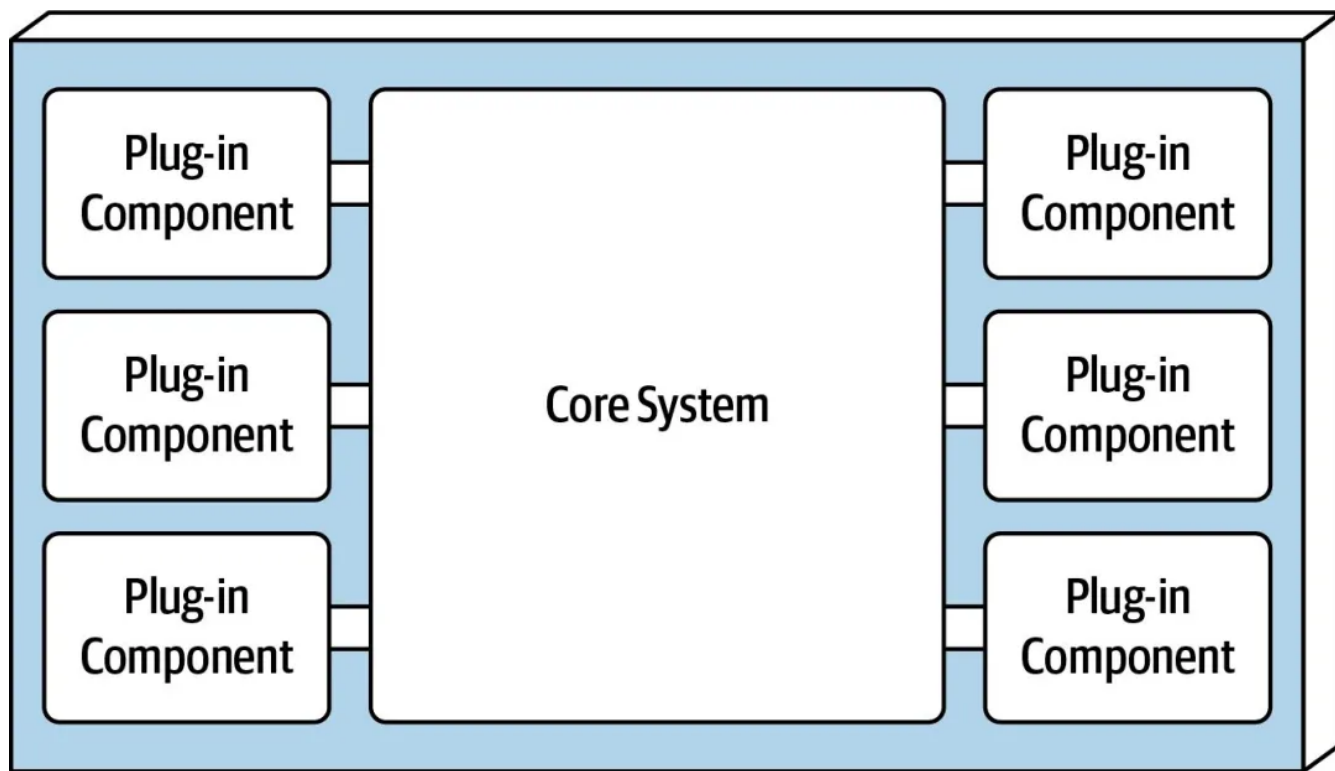
B/S结构利用不断成熟的WWW浏览器技术，结合浏览器的多种脚本语言，用通用浏览器就实现了原来需要复杂的专用软件才能实现的强大功能，并节约了开发成本。



在B / S结构中，除了数据库服务器外，应用程序以网页形式存放于Web服务器上，用户运行某个应用程序时只需在客户端上的浏览器中键入相应的网址(URL)，调用Web服务器上的应用程序并对数据库进行操作完成相应的数据处理工作，最后将结果通过浏览器显示给用户。

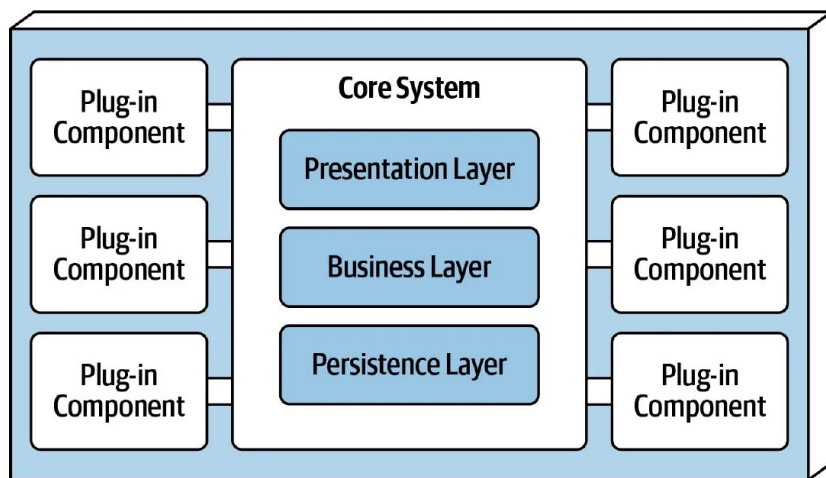
(8) 微内核 (Microkernel) 架构

也被称为插件式架构 (plug-in architecture) , 通常由以下两部分组成: **核心系统 (core system)** 和**插件 (plug-in component)** , 将应用系统的业务逻辑拆分成核心系统和插件, 能够提供很好的可扩展性和灵活性, 极大地方便了后续需求的新增和修改, 成功应用包括谷歌的Chrome浏览器, 微软的VS Code。

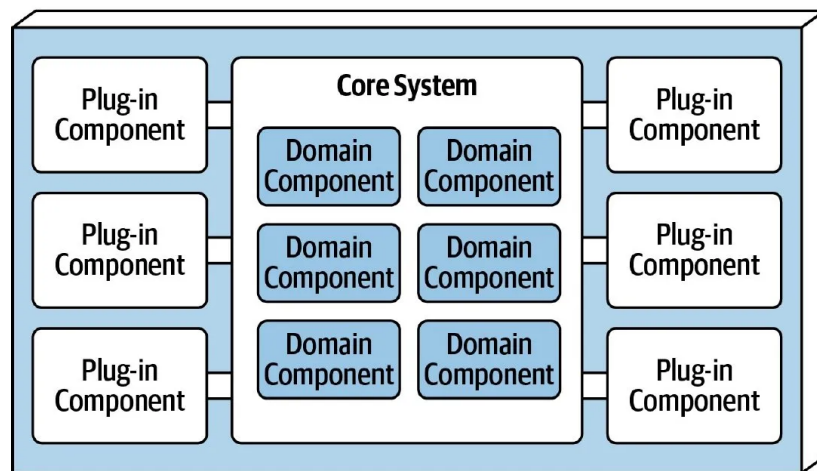


(8) 微内核 (Microkernel) 架构-----核心系统

核心系统通常只需提供能够支撑整个系统正常运行的基本功能，比如前文所举的VS Code例子，用户初始安装的是VS Code的核心系统，它只是一个提供了打开文件、编辑文件内容和保存文件等基本功能的文本编辑器，其他的扩展功能（如语法检查）都是通过安装插件集成的。**将复杂的业务逻辑从核心系统中剥离出来，并通过插件实现，能够提升系统的可扩展性和可维护性。**同时，因为复杂的功能都成了互不干扰的插件，系统的可测试性也得到了提高。



Layered Core System (Technically Partitioned)



Modular Core System (Domain Partitioned)

微内核架构在实现时通常都结合其他架构模式，这主要体现在核心系统的设计上，比如根据具体的业务特点，我们可以将核心系统设计成分层架构，或者是模块化架构。

(8) 微内核 (Microkernel) 架构-----插件

插件就是一些包含了定制化业务逻辑、扩展功能、附加功能的独立组件，用于扩充核心系统的功能。

插件之间是独立的，插件与核心系统之间则一般是“点对点”通信：核心系统通过调用插件提供的接口（比如插件类的方法）使用扩展功能。

插件可以划分为**编译时插件**和**运行时插件**两种类型，前者每次变更都需要重新构建和部署整个系统，但实现较为简单；后者则可以在系统运行时进行插件的新增和删除操作，相对地，实现也较为复杂。

编译时插件：在编译时插件中，插件通常以package或namespace实现，比如在package中可以以这样的命名规则来区分插件：app.plug-in.<domain>.<context>。

运行时插件：运行时插件中插件的实现通常是动态库的形式，比如.jar、.so、.dll文件。

3.4 框架 (Framework)

框架是为了构建完整的应用程序组织起来的一种程序框架结构。在组织过程中，常常是用一个抽象类来特化某项具体应用的行为，这个抽象类描述了该应用的控制流范型及其相关的类。

框架是特定于某一类应用的，框架中的类库一般也特定于应用，而不是普遍适用的。

复用框架，可以通过选择合适的子类来执行框架中的大部分操作，而不是从头开始编写子类的行为。

框架是一组相互协作的类，形成某类关键的一个可复用设计。框架将设计划分为一组抽象类，并定义它们各自的责任和相互之间的协作，以此来指导体系结构级的设计，开发这通过集成框架类中的类和组合其实例来定制该框架以生成特定的重用。

框架是半成品的软件，这是它和其他所有软件组件的本质区别。框架是一个可实例化的、部分完成的软件系统或子系统，它为一组系统或子系统定义了架构，并提供了构造系统的基本构造块，还为实现特定功能定义了可调整点。

框架内分可变部分和不可变部分。可变部分往往被封装为“扩展点”，扩展机制被纳入到框架的控制范围之内。

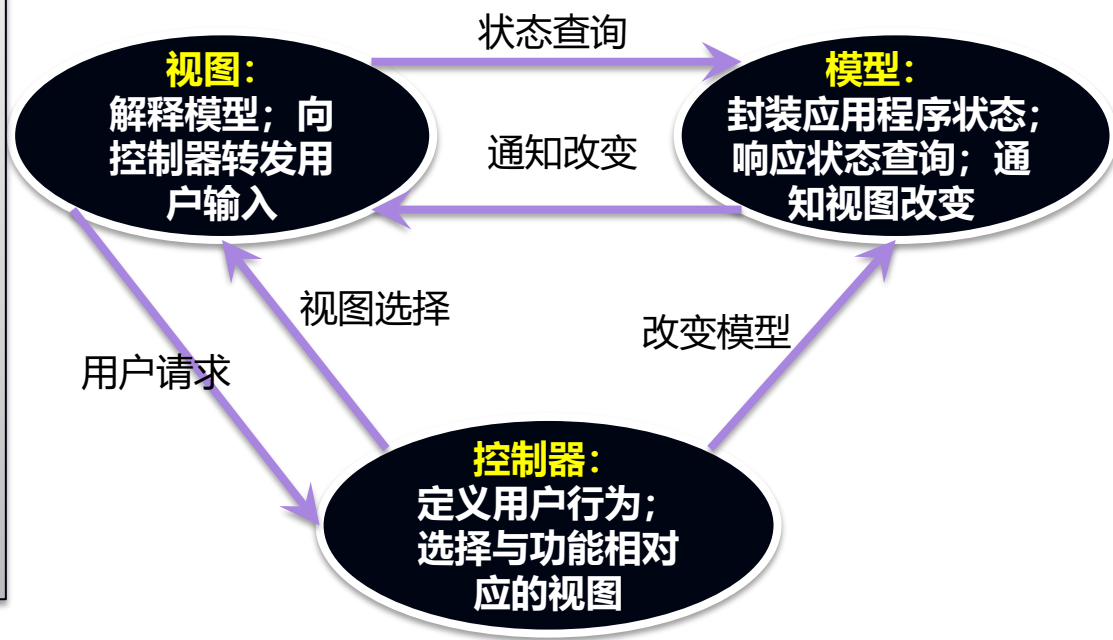
■ (1) 模型-视图-控制器 (MVC) 框架

最著名的体系结构框架之一是模型-视图-控制器 (MVC) 框架，它强调将用户输入、数据模型、数据表示的方式分开设计

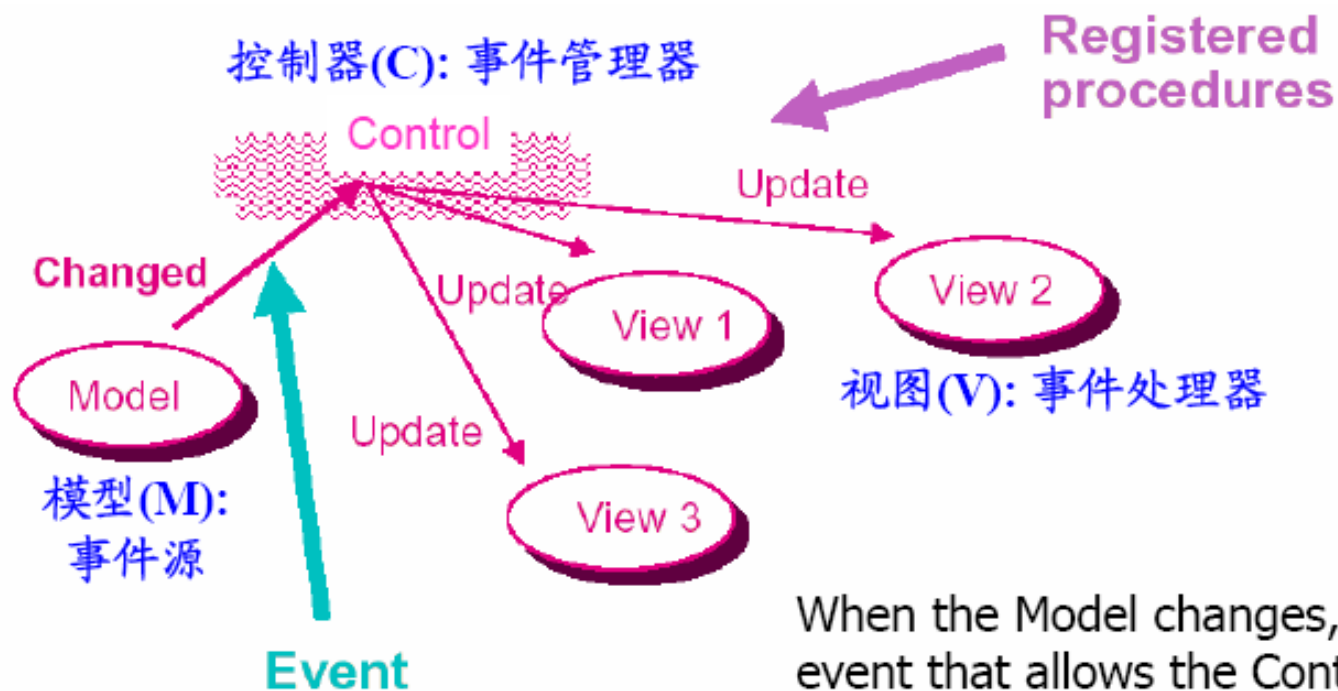
一个交互式应用系统由模型 (Model)、视图 (View) 和控制器 (Controller) 三个部分组成，分别对应于内部数据、数据表示和输入输出控制部分。

处理过程为：

首先控制器接收用户请求，并决定调用哪个模型来进行处理；然后模型用业务逻辑来处理用户的请求并返回数据；最后控制器用相应的视图格式化模型返回的数据，并通过表示层呈现给用户。其中，模型是核心数据和功能，视图只关心显示数据，控制器只关心用户输入，这种结构由于数据和业务规则同表示层分开，因此，可以最大化重用代码

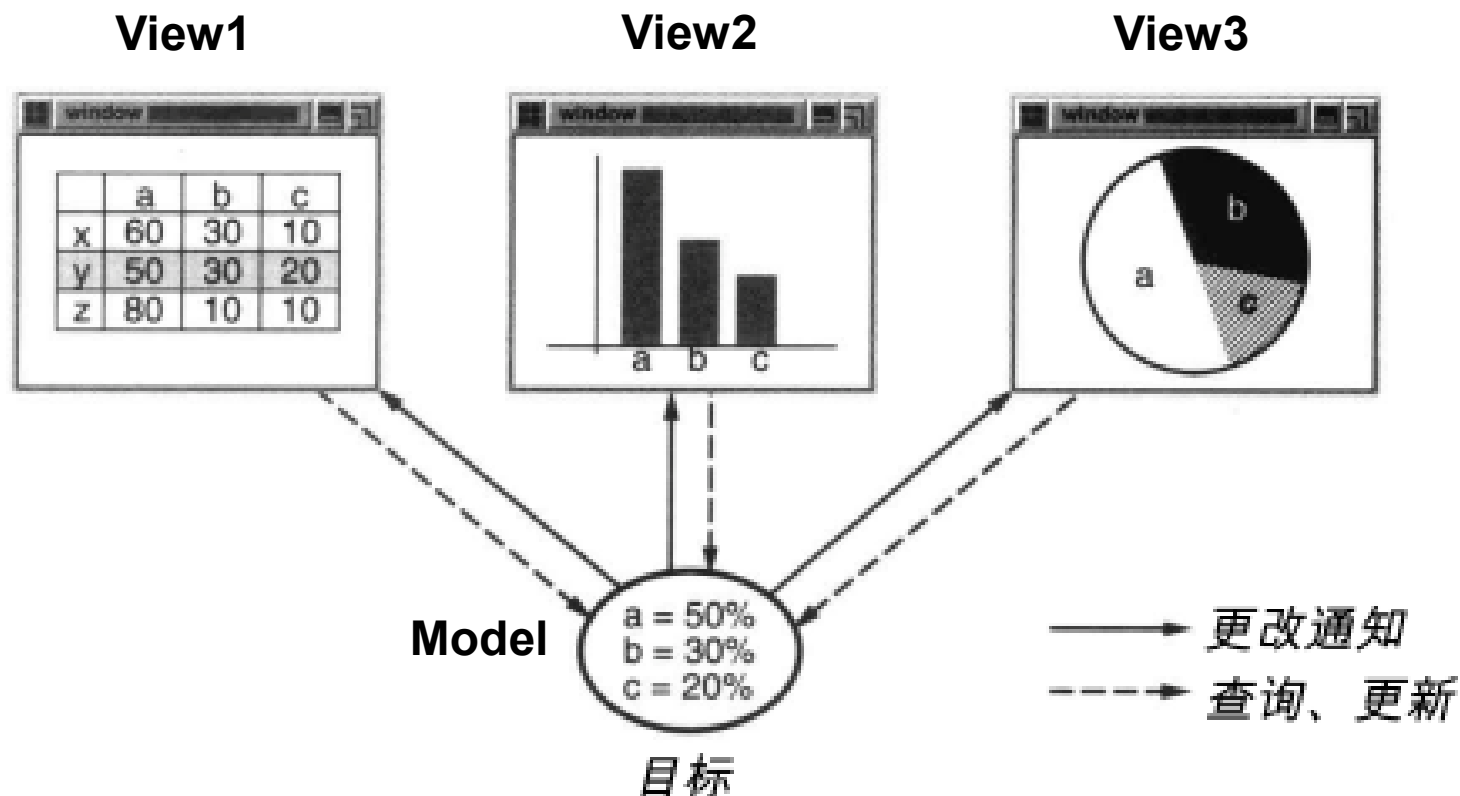


■ (1) 模型-视图-控制器 (MVC) 框架

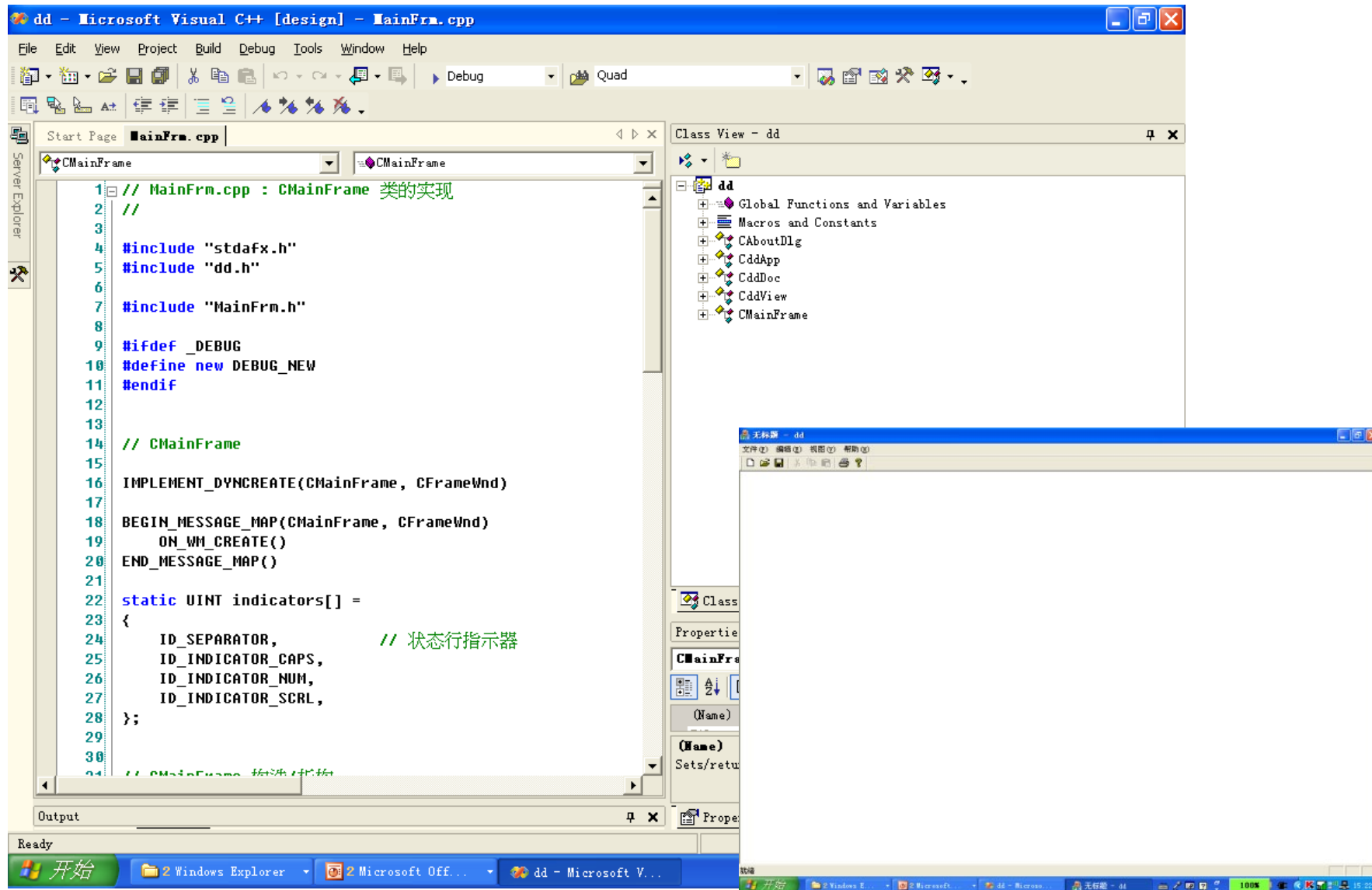


When the Model changes, it announces an event that allows the Controller to automatically invoke the procedures to update each View related to the Model. (当模型数据发生改变时, 它将发布事件, 控制器自动调用相关View的“更新”过程)

■ (1) 模型-视图-控制器 (MVC) 框架



03 面向对象设计之系统设计



■ (2) MEF (Managed Extensibility Framework) 框架

Managed Extensibility Framework (MEF) 是用于创建可扩展的轻量级应用程序的库。它让应用程序开发人员得以发现和使用扩展且无需配置。它还让扩展开发人员得以轻松地封装代码并避免脆弱的紧密依赖性。如果聊到MEF不得不提到的是IoC (IoC-Inversion of Control) , 即控制反转。它是一种程序设计指导思想。

(1) 依赖 (Dependency) : 表示有关联, 一个类依赖于另一个类。在日常编码的时候大家追求的都是高内聚低耦合这种就是良性的依赖; 常说的代码牵一发动全身的则是恶性依赖重则推到重构、轻则维护困难。

(2) 依赖倒置原则 (DIP) : 高层模块不应依赖于底层模块, 两者应该依赖于抽象。抽象不应该依赖于细节, 细节应该依赖于抽象。

(3) 控制反转 (IoC) : 上层对下层的依赖 (即底层模块的获得) 交给第三方。例如在WPF里常用的MVVM模式也会涉及到这样的概念, 在ViewModel层想调用View的窗体关闭这个时候VM层没有办法直接调用, VM层又不知道自己什么时候会被初始化 (View层)。这个时候就可以用控制反转的思维, 让View层决定什么时候初始化ViewModel再将关闭View的权利转交给ViewModel。

(4) 依赖注入 (DI) : 实现IoC的一种方式、手段。将标记了Export的特性的dll注册到IoC容器中。

■ (2) MEF (Managed Extensibility Framework) 框架

契约 (Contract)

- 定义：插件与宿主程序之间的约定，它规定了插件必须实现的功能方法。
- 作用：确保类型安全和规范接口

部件 (Part)

- 定义：部件是 MEF 中可组合的最小单位，插件即是一个部件
- 作用：- 实现功能模块化 - 支持动态扩展

目录 (Catalog)

- 定义：部件的容器和发现机制
- 作用：自动扫描和发现插件、支持多种加载方式、管理插件的元数据

组合 (Composition)

- 定义：部件的装配和依赖解析过程，将导出和导入匹配并组装在一起
- 作用：自动解析依赖关系、处理部件生命周期、管理部件实例化

元数据 (Metadata)

- 定义：部件的描述信息
- 作用：提供额外的配置和筛选能力

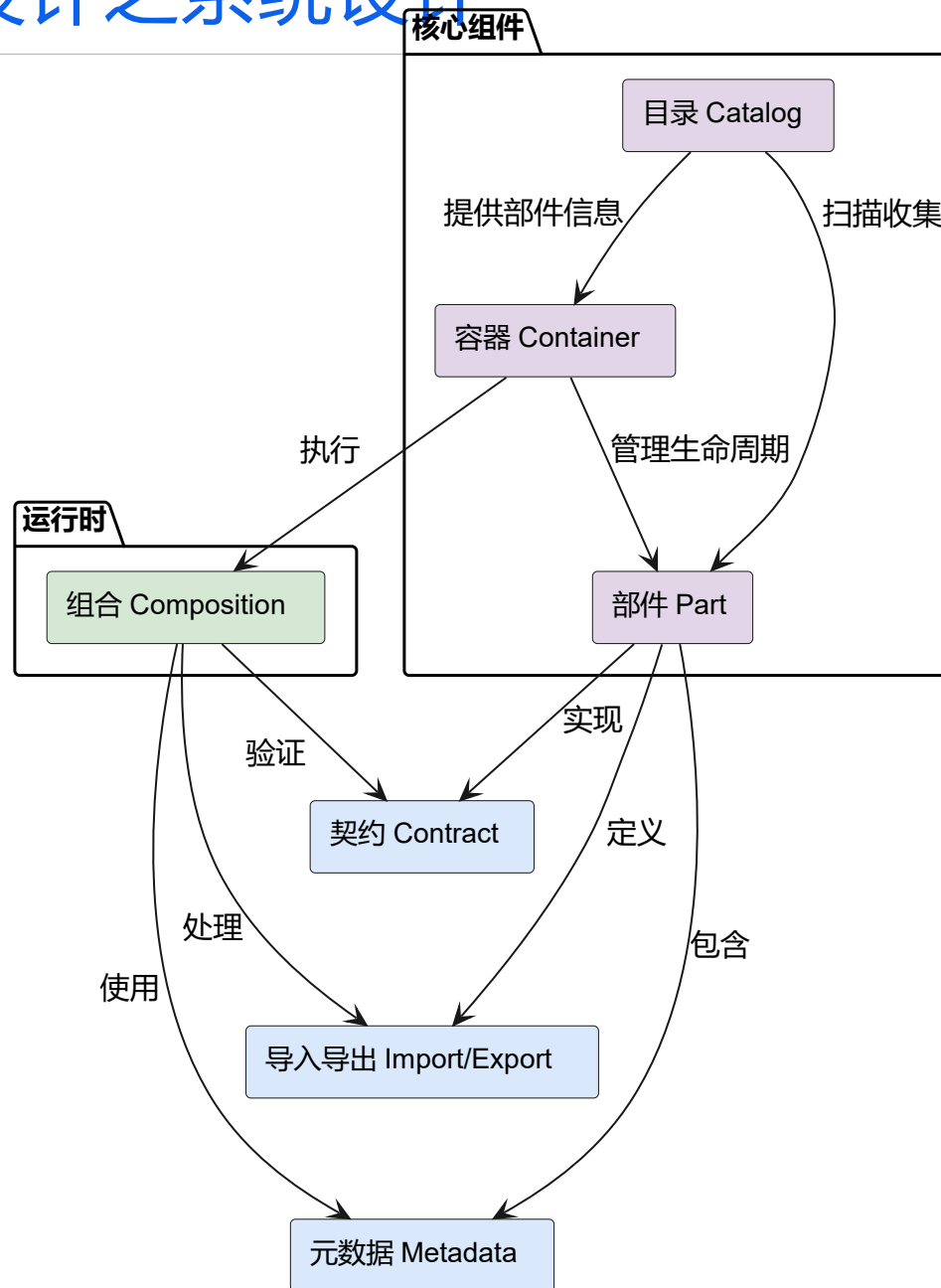
导入导出 (Import/Export)

- 定义：部件间的连接机制和依赖声明
- 作用：[Export]向其他组件提供功能
[Import]导入其他组件的功能

容器 (Container)

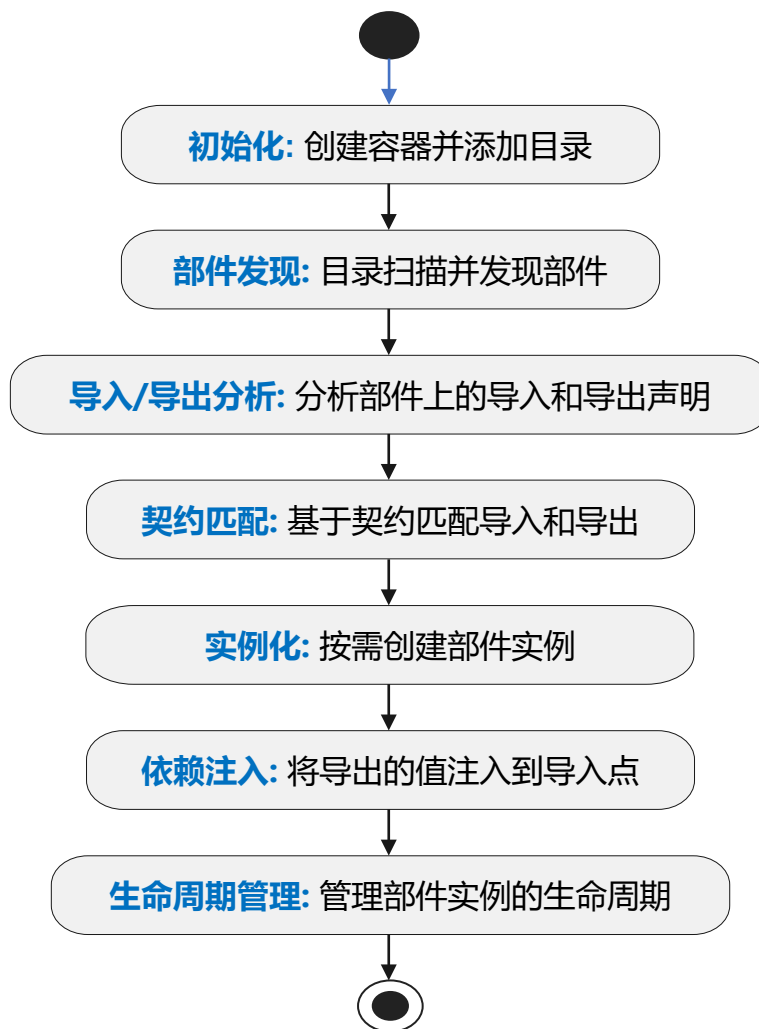
- 定义：部件的运行环境
- 作用：管理生命周期和依赖注入

MEF框架相互关系

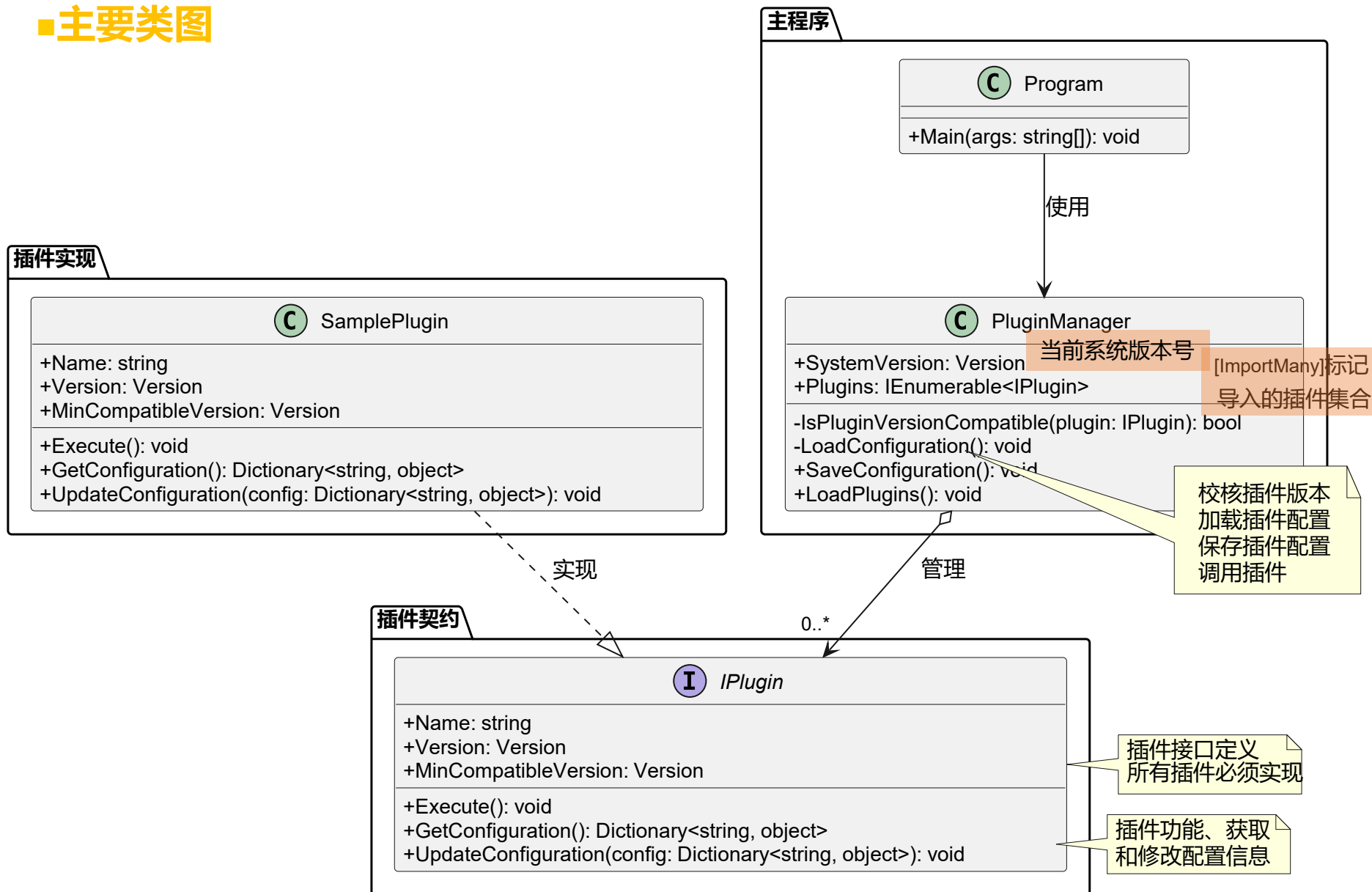


■ 基本功能——插件的加载和执行

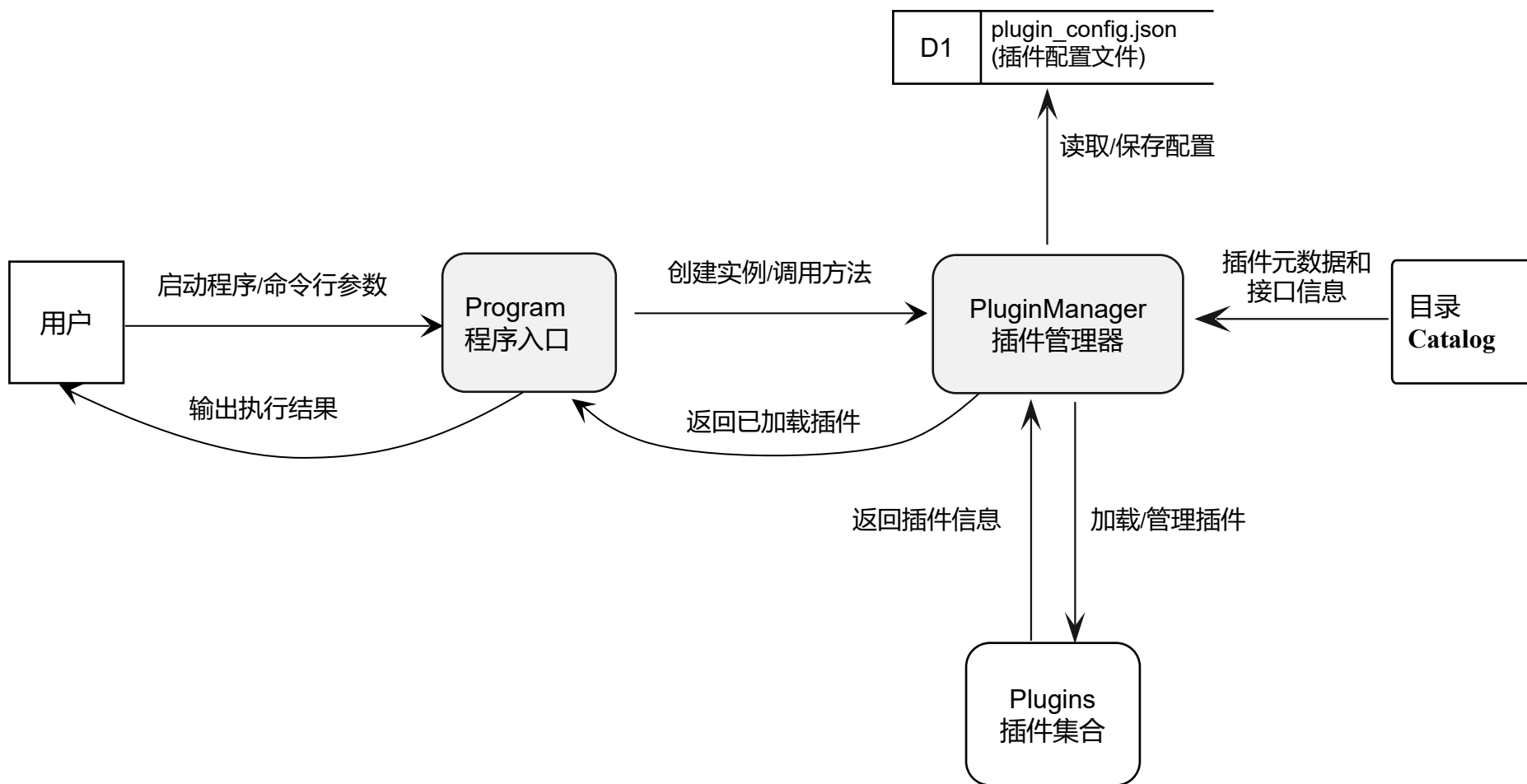
实现一个基础的插件式软件的基础功能大致需要这些步骤：



■ 主要类图



■ 数据流图



3.5 设计模式

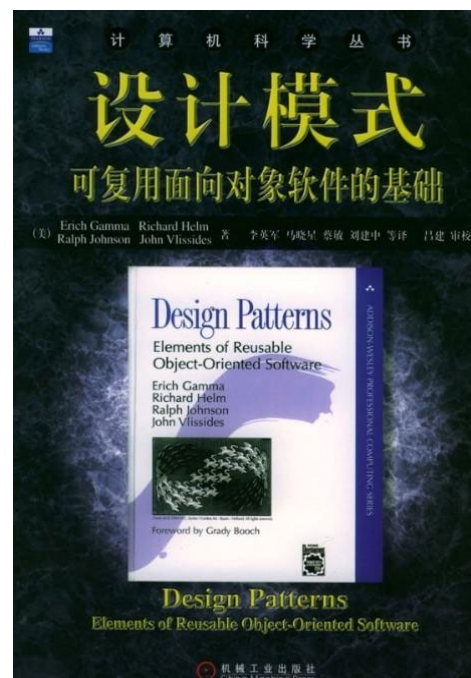
模式是一种业已验证的通用问题解决方案。

模式复用：在所要开发的系统中，使用现有的模式。

而不是从头设计解决方案。对于经典的模式，一般有模式指南，模式指南说明了在什么情况下，可以使用模式，以及使用模式的各种用法及其优缺点。

复用模式的优点：

- 模式是已经被其他人仔细思考过，并在以往的问题中得到了应用，与没有经过测试、定制的方法相比，模式会更加正确，更为健壮。
- 在使用模式进行程序实现时，仍然可以使用我们熟悉的语言。
- 有许多文献对各种不同的模式进行了仔细分析，解释了他们之间的细节差异。
- **设计模式—可复用的面向对象软件的基础**（美）Eric Gamma 等著，该书详细分析了20多种设计模式。

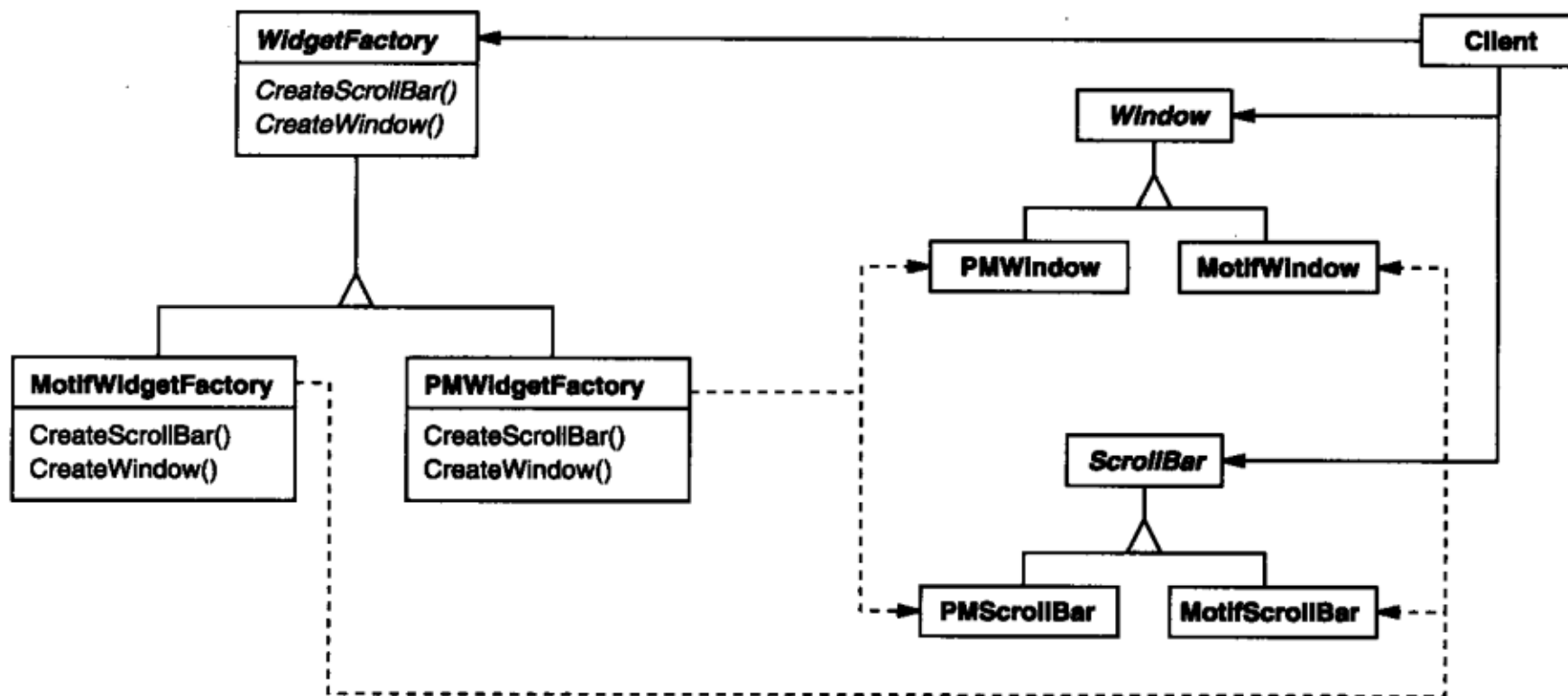


一般来说，一个模式有4个要素：

- (1) 模式名称：说明模式问题、解决方案和效果
- (2) 问题：说明在何种场合使用该模式。要描述使用模式的先决条件和特定设计问题。模式的适用条件
- (3) 解决方案：描述设计的组成成分、它们之间的关系、各自的职责和合作方式。由于模式就像一个模板，可应用于多种不同的情况，所有解决方案并不描述一个特定而具体的设计或实现，而是提供设计问题的抽象描述和怎样用一个一般意义的元素组合（类或对象的组合）来解决这个问题
- (4) 效果：描述了模式使用效果及使用模式应该权衡的问题。模式使用的效果对于评价设计选择和理解使用模式的代价及好处具有重要意义。

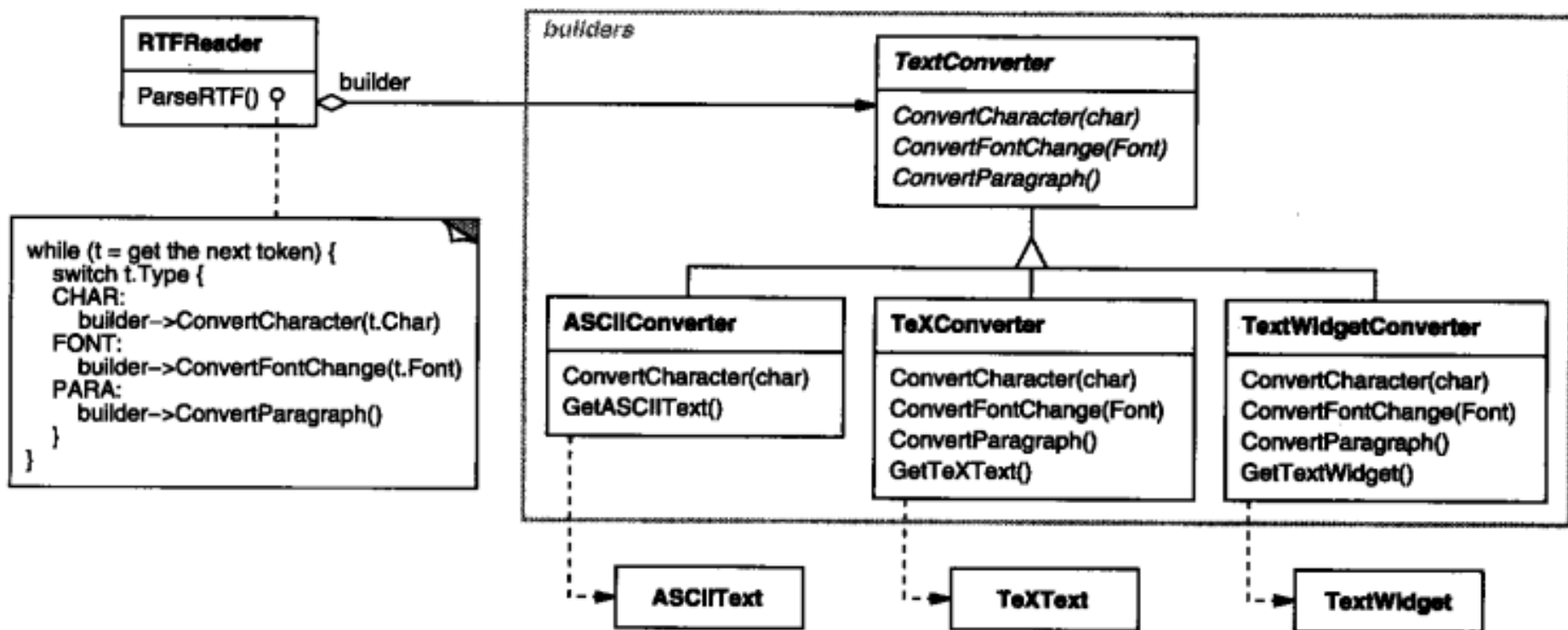
ABSTRACT FACTORY (抽象工厂) — 对象创建型模式

提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们具体的类。



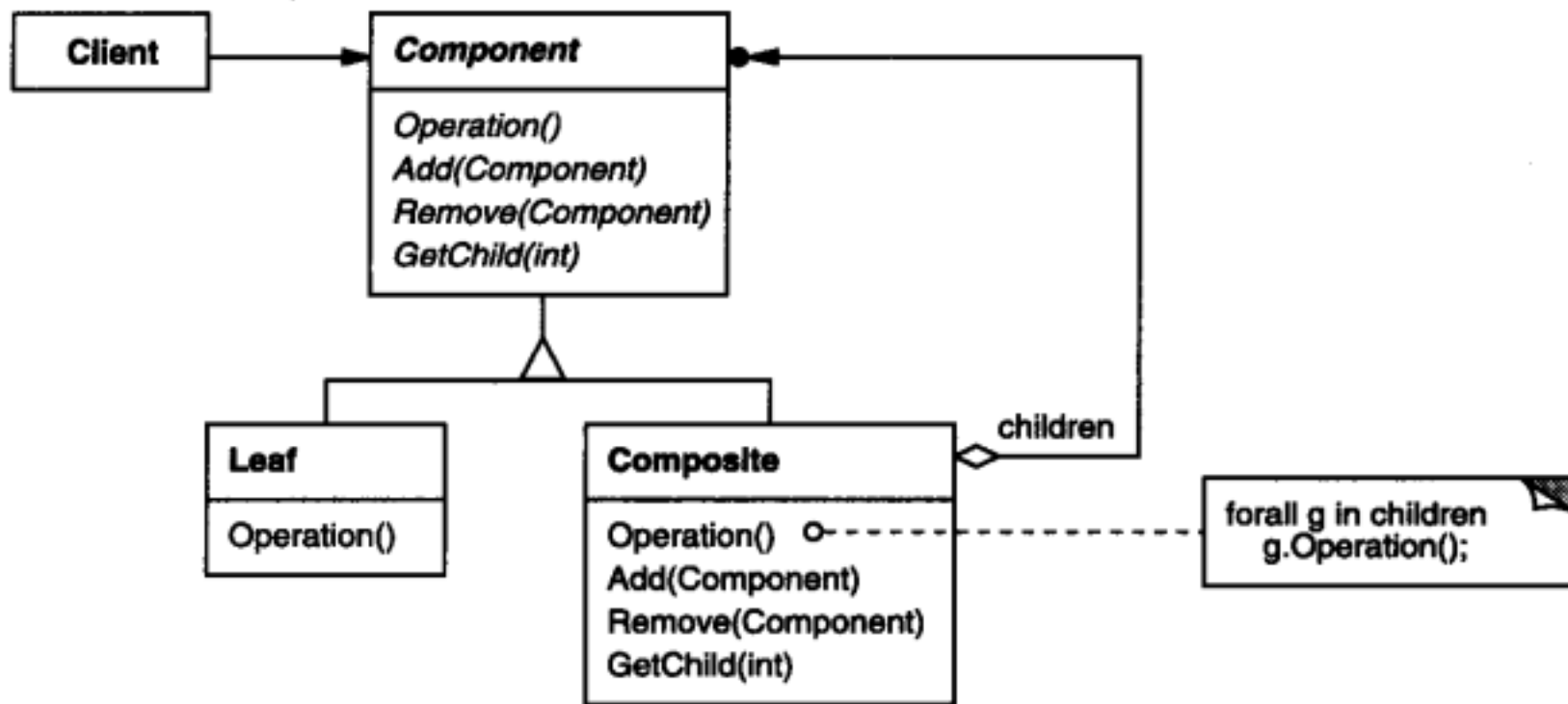
BUILDER（生成器）—对象创建型模式

将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。

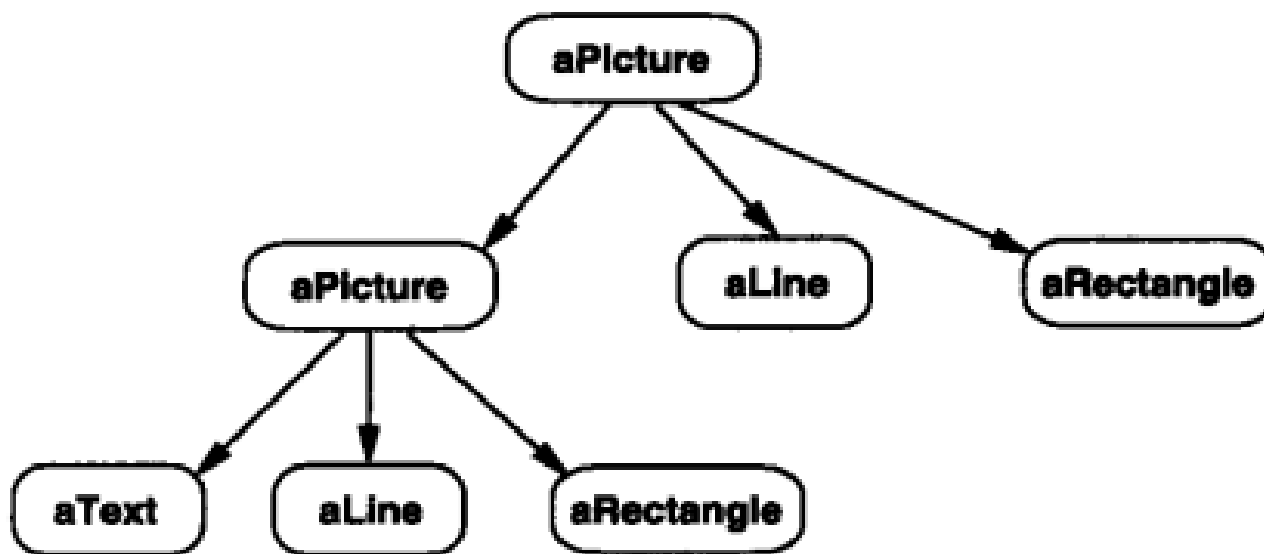


COMPOSITE（组合）—对象结构型模式

- ❑ 将对象组合成树形结构以表示“部分 - 整体”的层次结构。
- ❑ Composite使得用户对单个对象和组合对象的使用具有一致性。

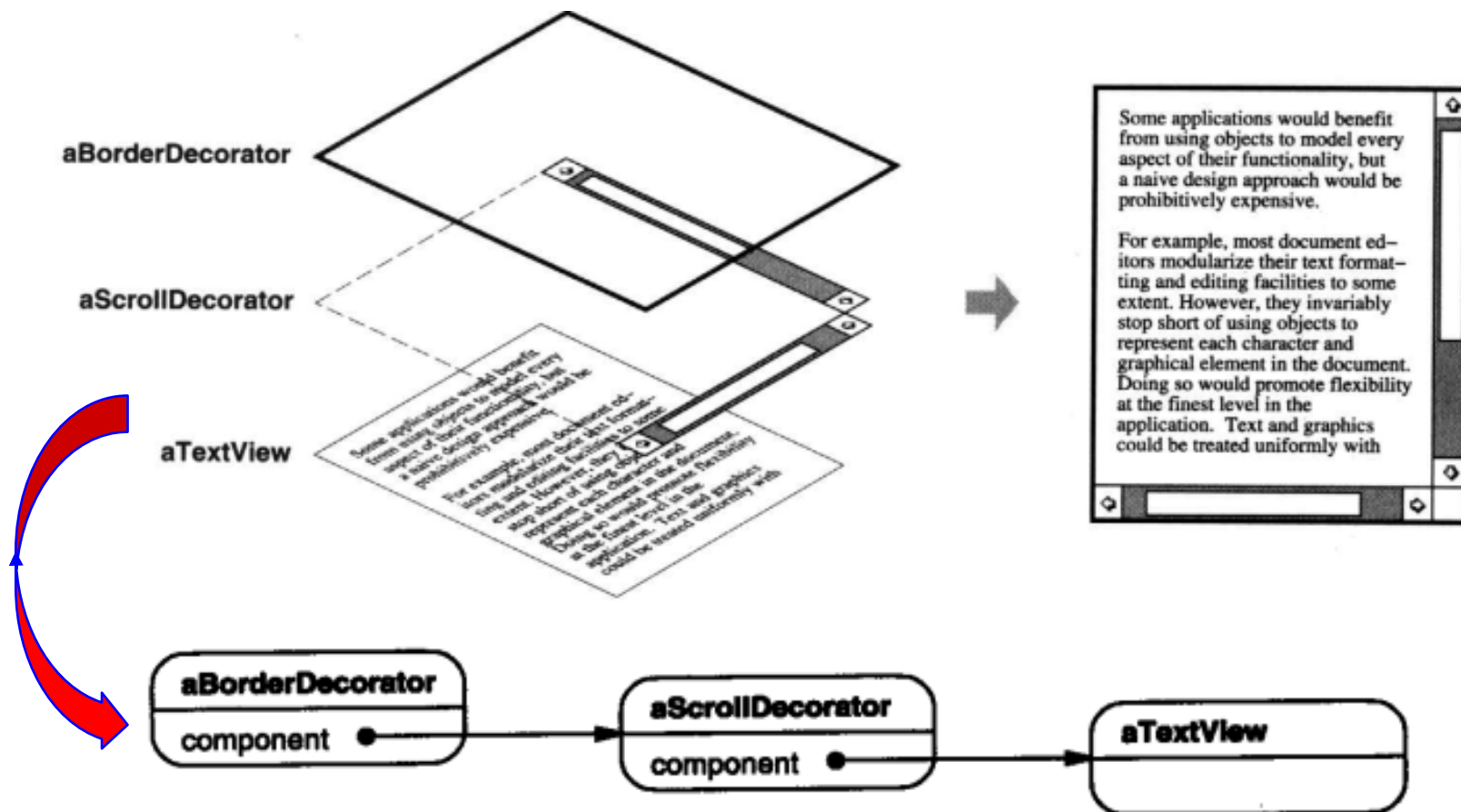


COMPOSITE（组合）—对象结构型模式

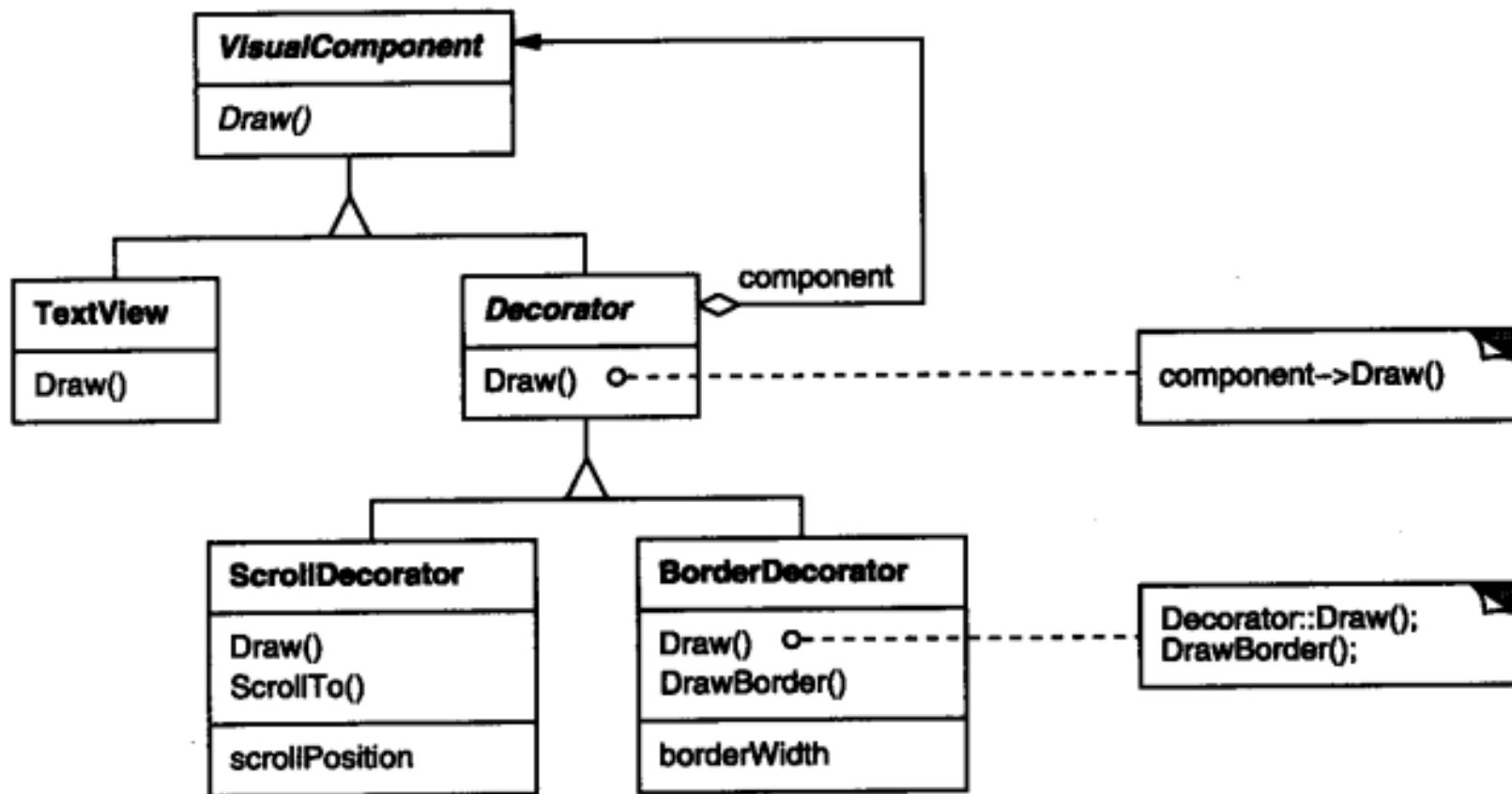


DECORATOR (装饰) — 对象结构型模式

动态地给一个对象添加一些额外的职责。就增加功能来说，Decorator模式相比生成子类更为灵活。

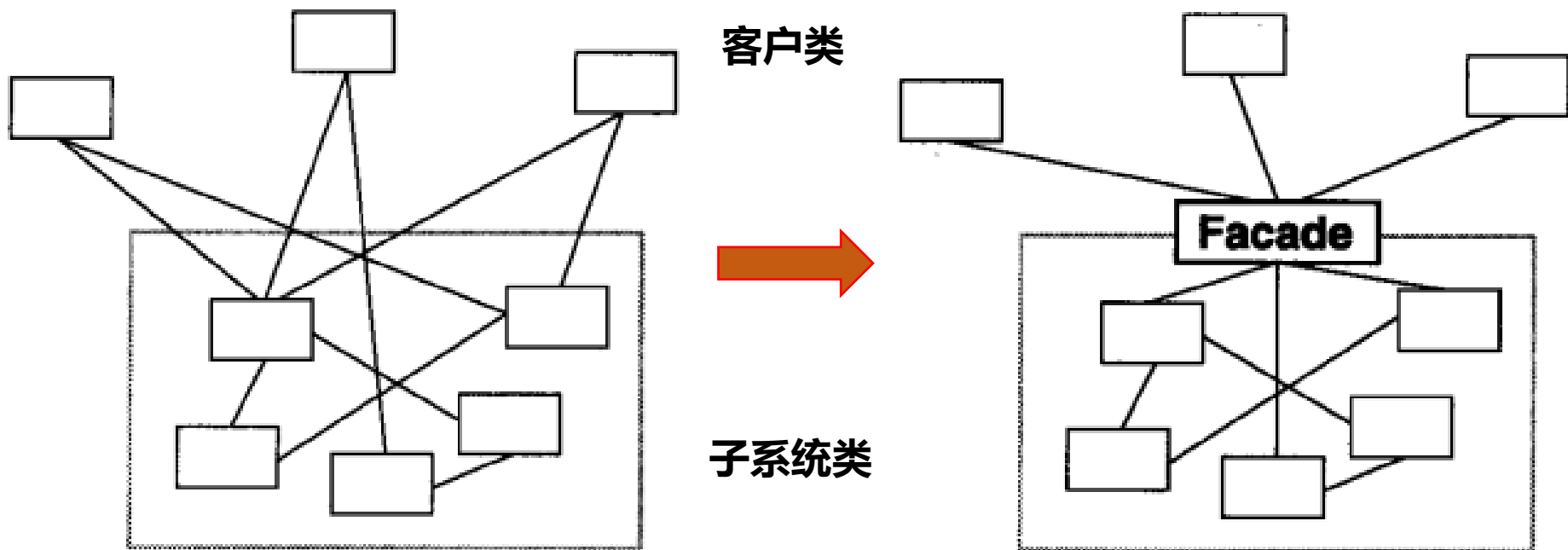


DECORATOR (装饰) —对象结构型模式

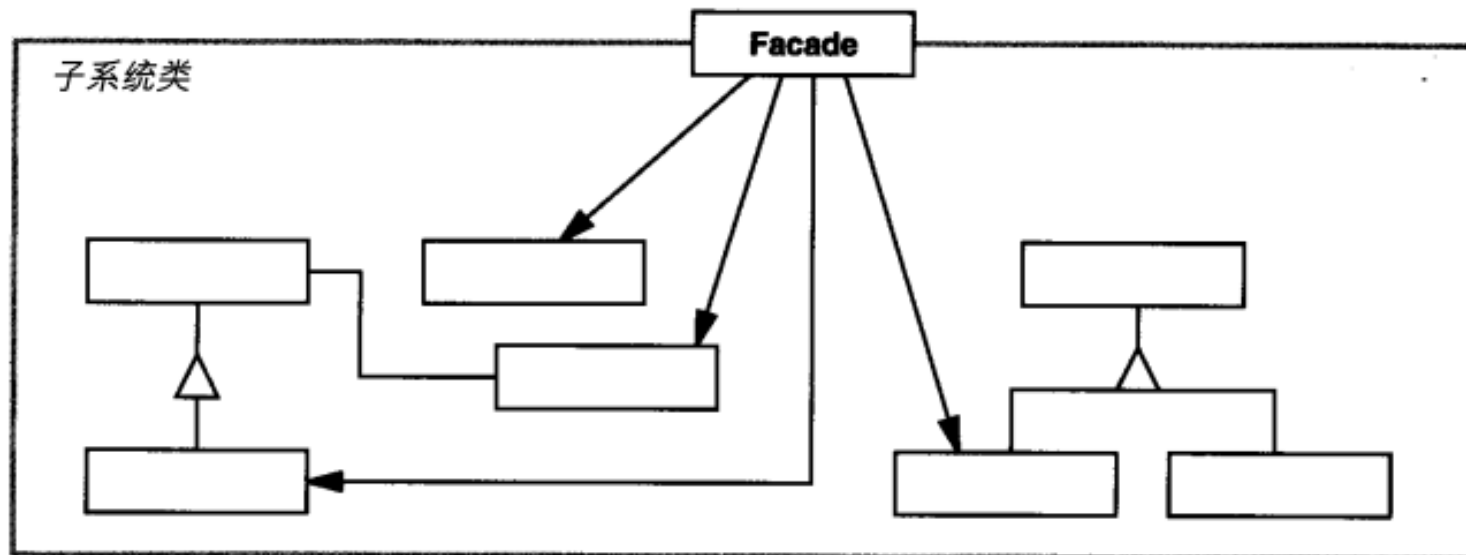


FACADE（外观）—对象结构型模式

为子系统的一组接口提供一个一致的界面， Facade模式定义了一个高层接口，这个接口使得这一子系统更加容易使用。



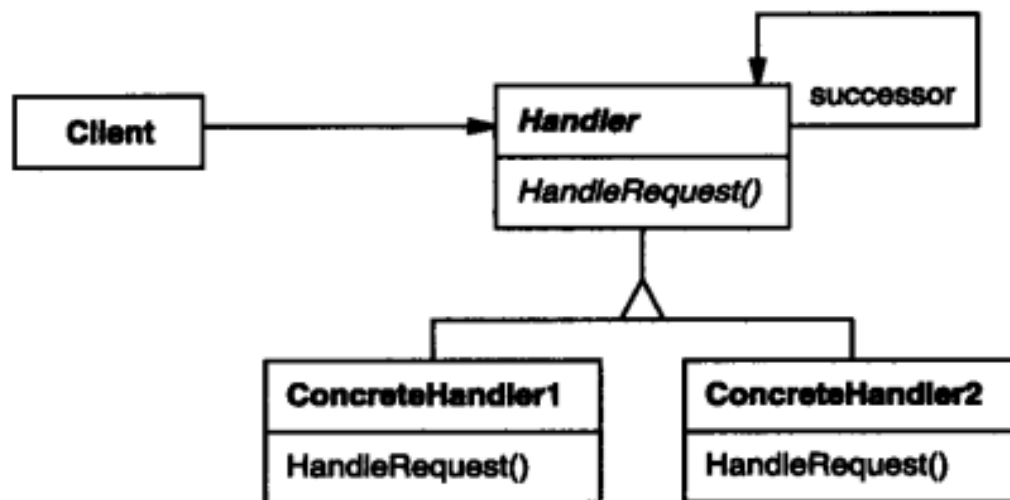
FACADE（外观）—对象结构型模式



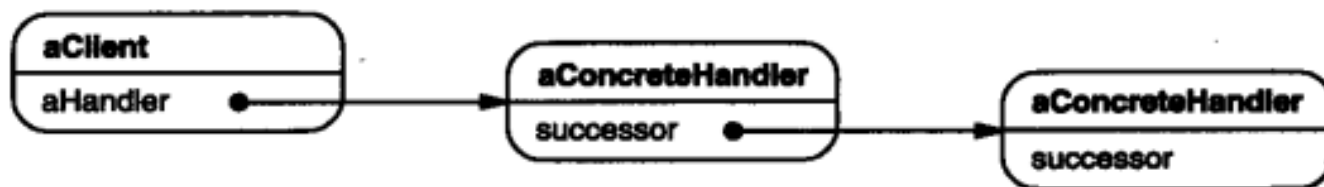
- Facade (Compiler)
 - 知道哪些子系统类负责处理请求。
 - 将客户的请求代理给适当的子系统对象。
- Subsystem classes (Scanner、Parser、ProgramNode等)
 - 实现子系统的功能。
 - 处理由Facade对象指派的任务。
 - 没有facade的任何相关信息；即没有指向facade的指针。

CHAIN OF RESPONSIBILITY(职责链)

- 使多个对象都有机会处理请求，从而避免请求的发送者和接收者之间的耦合关系。
- 将这些对象连成一条链，并沿着这条链传递该请求，直到有一个对象处理它为止。

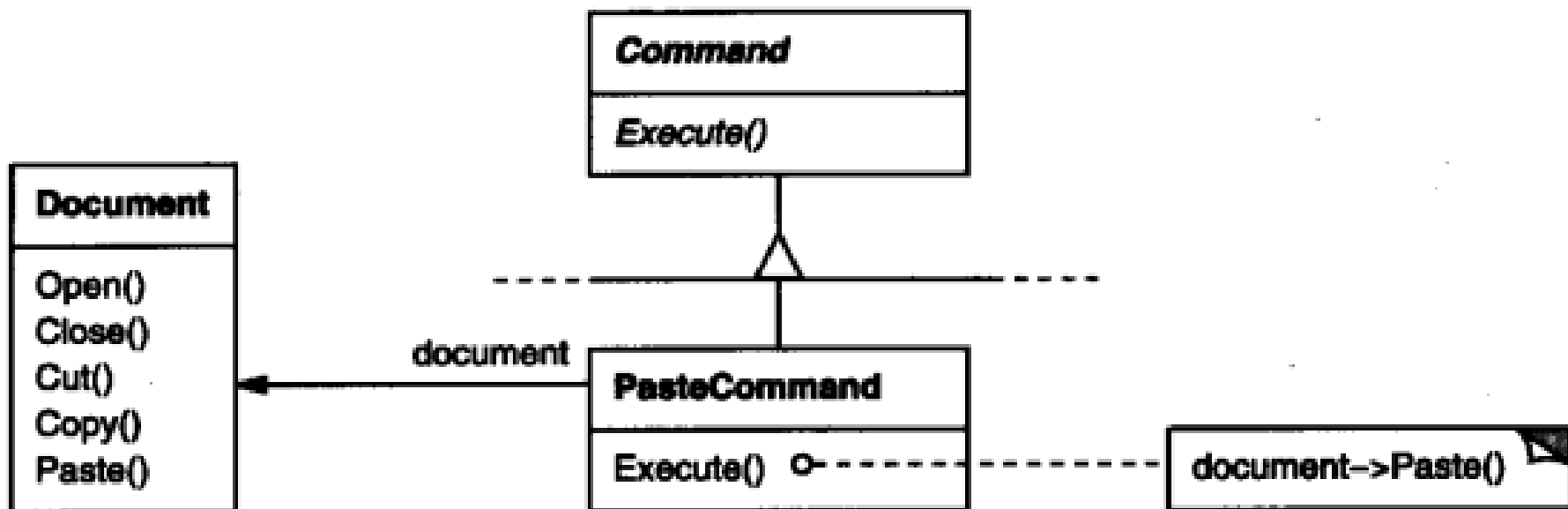


一个典型的对象结构可能如下图所示：



COMMAND (命令)

- 将一个请求封装为一个对象，从而使你可用不同的请求对客户进行参数化；
- 对请求排队或记录请求日志，以及支持可撤消的操作。



04

面向对象设计的准则

4.1 模块化

面向对象软件开发很自然地支持了把系统分解为模块的设计原理：对象就是模块。它是把数据结构和操作这些数据的方法紧密地结合在一起所构成的模块。

4.2 抽象

面向对象方法不仅支持过程抽象，而且还支持数据抽象。类实际上是一种抽象数据类型，它对外开发的公共接口构成了类的规格说明，这种接口规定了外界可以使用的合法操作符，利用这些操作符可以对类实例中包含的数据进行操作。使用者无须知道这些操作符的实现算法和类中的数据元素的具体表示方法，就可以通过这些操作符使用类中定义的数据。通常把这类抽象称为规格说明抽象。

4.3 信息隐藏

在面向对象方法中，信息隐藏通过对象的封装性实现：类结构分离了接口与实现，从而支持了信息隐藏。对类的用户来说，属性的表示方法和操作的实现算法都应该是隐藏的。

4.4 弱耦合

耦合是指一个软件结构内不同模块之间互联的紧密程度。在面向对象方法中，对象是最基本的模块，因此，耦合主要指不同对象之间相互关联的紧密程度。

弱耦合是优秀软件设计的一个重要标准，因为这有助于使得系统中某一部分的变化对其他部分的影响降到最低程度。在理想情况下，对某一部分的理解、测试或修改，无须涉及系统的其他部分。

如果一类对象过多地依赖其他对象类完成自己的工作，则不仅给理解、测试或修改这个类带来很大困难，而且还将大大降低该类的可重用型和可移植性。

对象不可能是完全孤立的，当两个对象必须相互联系相互依赖时，应该通过类的协议（公共接口）实现耦合，而不应该依赖类的具体实现细节。

面向对象设计方法中主要存在以下两种对象耦合形式：

- ◆ **交互耦合**：对象之间的耦合通过消息连接来实现，为了尽可能松散，应该遵守如下准则：
 - 尽量减低消息连接的复杂程度：应该尽量减少消息中包含的参数个数，降低参数的复杂程度
 - 减少对象发送（或接收）的消息数
- ◆ **继承耦合**：与交互耦合相反，应该提高继承耦合程度。继承是“一般与特殊”之间耦合的一种形式。从本质上看，通过继承关系结合起来的基类和派生类，构成了系统中粒度更大的模块。因此，它们之间彼此应该结合的越紧越好。

4.5 强内聚

内聚衡量一个模块内各个元素彼此结合的紧密程度。设计中使用的一个构件内的各个元素，对完成一个定义明确的目的所做出的贡献程度。在设计时应该力求做到高内聚。

在面向对象设计中存在且仅存在下述3种内聚：

服务内聚：一个服务应该完成一个且仅完成一个功能；

类内聚：一个类应该只有一个用途，它的属性和服务应该是高内聚的。类的属性和服务应该全部是完成该类对象的任务所必需的，其中不包含无用的属性或服务。如果某个类有多个用途，通常应该把它分解成多个专用的类。

一般-特殊内聚：设计出的一般-特殊结构，应该符合多数人的概念，更准

一般-特殊结构的深度要适当：应该使类等级中包含的层次数适当。一般说来，在一个中等规模(大约包含100个类)的系统中，类等级层次数应保持为 7 ± 2 。不应该仅仅从方便编码的角度出发随意创建派生类，应该使一般-特殊结构与领域知识或常识保持一致。

4.6 可重用

软件重用是提高软件开发生产率和目标系统质量的重要途径。重用基本上从设计阶段开始。

重用有两个方面的含义：

一方面尽量使用已有的类（包括开发环境提供的类库，及以往开发类似系统时创建的类）；

二是如果确实需要创建新类，则在设计这些新类的协议时，应该考虑将来的可重用性。



05

对象设计

面向对象设计的两个关键步骤是：**完成类图设计和详细设计**。完成类图设计即确定属性的格式以及将方法分配给相关的类。

● 确定类的属性的格式

属性的格式基本上可以从分析结果中直接得出，决定属性格式的信息可以在分析流中获得，可以在分析 workflows 中将其加入到类图中。然而，面向对象范型是迭代的。每一次迭代都可能改变原有的设计，如果属性过早地加入到了类图中，这些属性也可能随之修改、或者从一个类中移动到另一个类，增加了一些不必要的工作。所以，晚一些将属性的信息加入到类图中还是有一些好处的。

● 为类分配方法

- 通过检查每一个场景的交互图来确定产品的所有操作
- 确定哪些方法属于哪些类

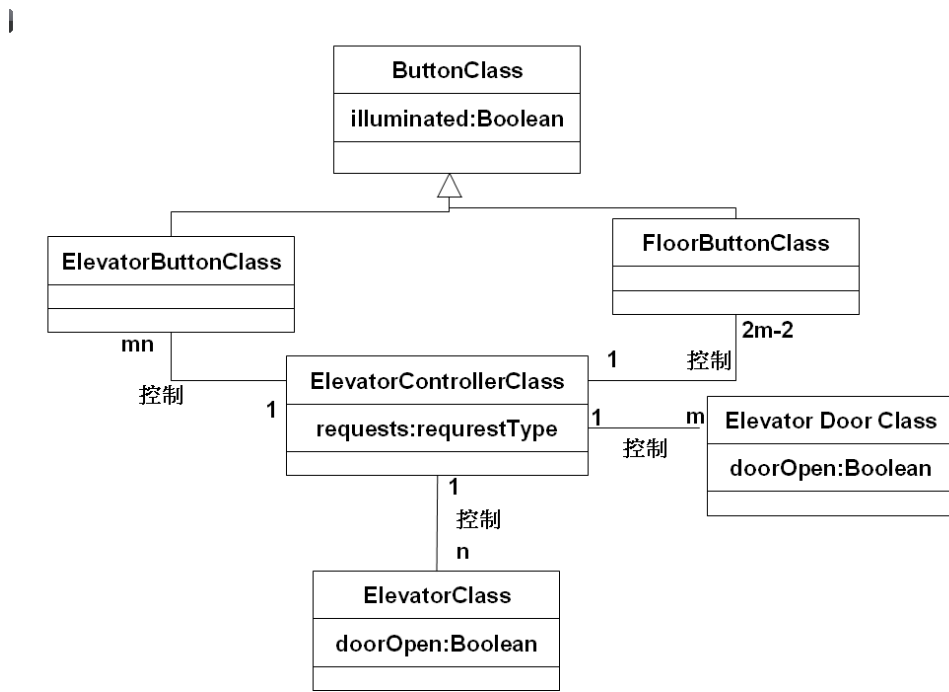
对象的客户是发送消息给给对象的程序单元

- 当需要发送消息给某个类或者客户，可以将方法分配给这个类或客户
- 如果方法操作的变量是某个类的状态变量时，往往将该方法分配给此类
- 如果一个操作被一个对象的不同客户所调用，那么将这个操作实现为该对象的一个方法比在每一客户中实现它要有意义
- 如果一个客户发送一个消息给一个对象，请求该对象为之服务，该对象有责任实现此客户请求的每个细节。

● 电梯问题案例研究

1、启动定时器、检查请求集、更新请求集与其它类没有关系，它是类Elevator Controller Class的职责，按照职责驱动的原则，这三个方法应该分配给类Elevator Controller Class

2、其余8项职责，都是发送消息给其它类来通知它执行某些任务。根据前面所说的分配方法，这些职责的实现应该分配给接受消息的类。这样既符合职责驱动原则，又符合信息隐藏原则。



10 电梯问题案例研究类图的第三次迭代

类
Evevator Controller Class
职责
1. 发送消息给Elevator Button Class，使打开按钮 2. 发送消息给Elevator Button Class，使关上按钮 3. 发送消息给Floor Button Class，使打开按钮 4. 发送消息给Floor Button Class，使关上按钮 5. 发送消息给Elevator Class，使向上移动一层 6. 发送消息给Elevator Class 使向下移动一层 7. 发送消息给Elevator Doors Class 使打开门 8. 启动定时器 9. 在超时后发送消息给Elevator Doors Class，使关闭门 10. 检查请求集 11. 更新请求集
协作
1. Evelator Button Class（子类） 2. Floor Button Class（子类） 3. Elevator Doors Class 4. Elevator Class

CRC卡：类职责协作卡

Elevator Controller Class
request: requestType
checkRequests updateRequests startTimer

8. 启动定时器
10. 检查请求集
11. 更新请求集

Elevator Class
moveDownOneFloor moveUpOneFloor

5. 发送消息给Elevator Class, 使向上移动一层
6. 发送消息给Elevator Class 使向下移动一层

Elevator Button Class
turnOffButton turnOnButton

1. 发送消息给Elevator Button Class, 使打开按钮
2. 发送消息给Elevator Button Class, 使关上按钮

Elevator Doors Class
doorOpen: boolean
closeDoors openDoors

7. 发送消息给Elevator Doors Class 使打开门
9. 在超时后发送消息给Elevator Doors Class, 使关闭门

Floor Button Class
turnOffButton turnOnButton

3. 发送消息给Floor Button Class, 使打开按钮
4. 发送消息给Floor Button Class, 使关上按钮

Button Class
illuminated: Boolean
turnOffButton(abstract) turnOnButton(abstract)

Button Class的两个子类都有相同名称的方法, 利用多态和动态绑定的理由, 该两方法在基类中声明为抽象的方法。

多态:

所谓多态，就是具有许多形态，指在继承关系中，子类对象可以象父类对象那样使用，同样的消息既可以发送给父类对象也可以发送到子类对象。也就是说，在类结构的不同层次中可以共享（公用）一个行为（方法）的名字，然而，不同层次中的各个类却各自按各自的需要来实现这个行为，即各个类接受消息的接口是一样的，但是，其实现各不相同。

***Button Class*的两个子类都有相同名称的方法，利用多态和动态绑定的基础是这两方法在基类中声明为抽象的方法。**

动态绑定:

当ButtonClass的某个实例收到消息时，它根据其是Elevator Button还是Floor Botton决定是调用Elevator Button Class的方法还是Floor Button Class的方法。而这个工作在编译时是不能完成的，必须在程序运行时才能确定，所以称为动态绑定。

Button Class
illuminated: Boolean
turnOffButton(abstract) turnOnButton(abstract)

2026

THANK YOU

浙江大学机械工程学院航空制造工程研究所 王青